

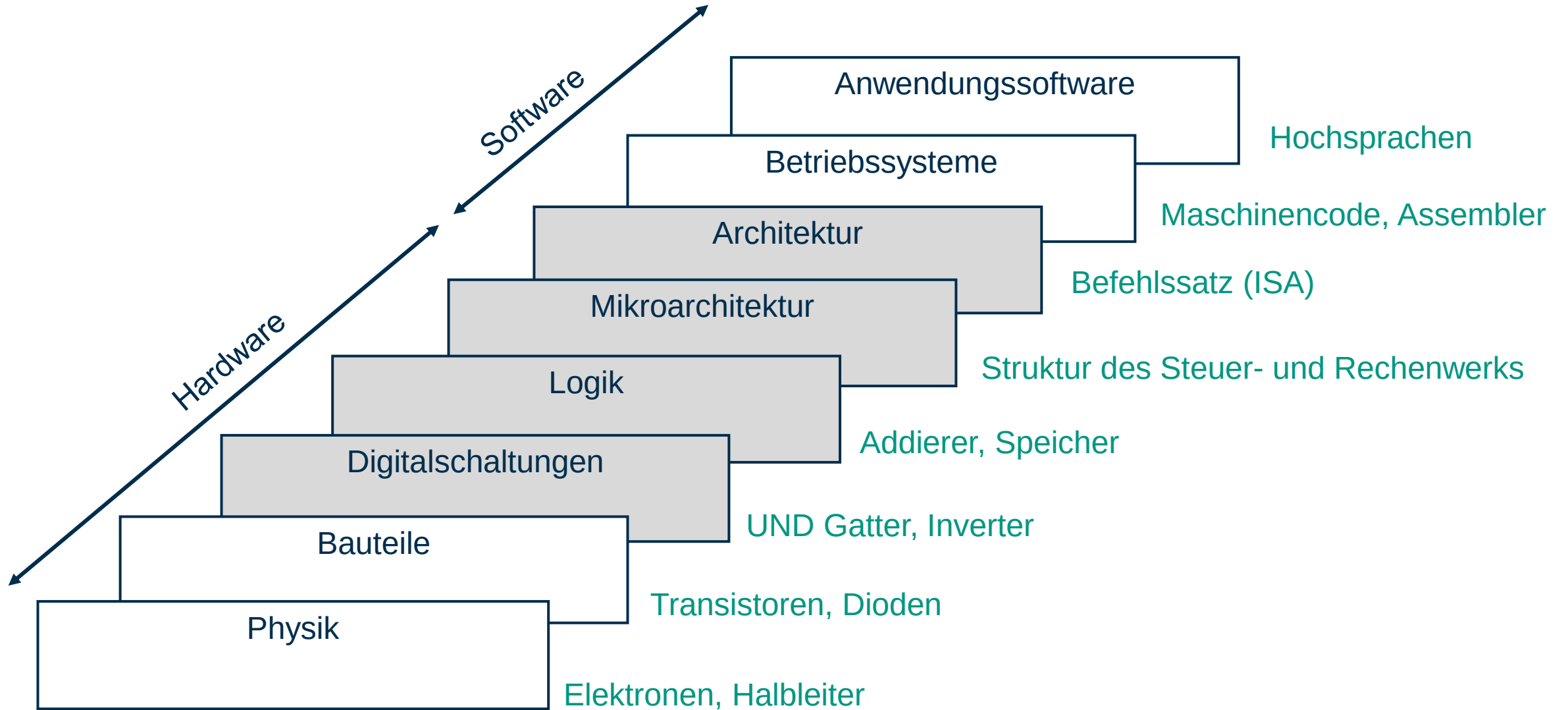
Informations verarbeitung

Digitaltechnik und Rechnerarchitektur

Dr.-Ing. Tanja Harbaum

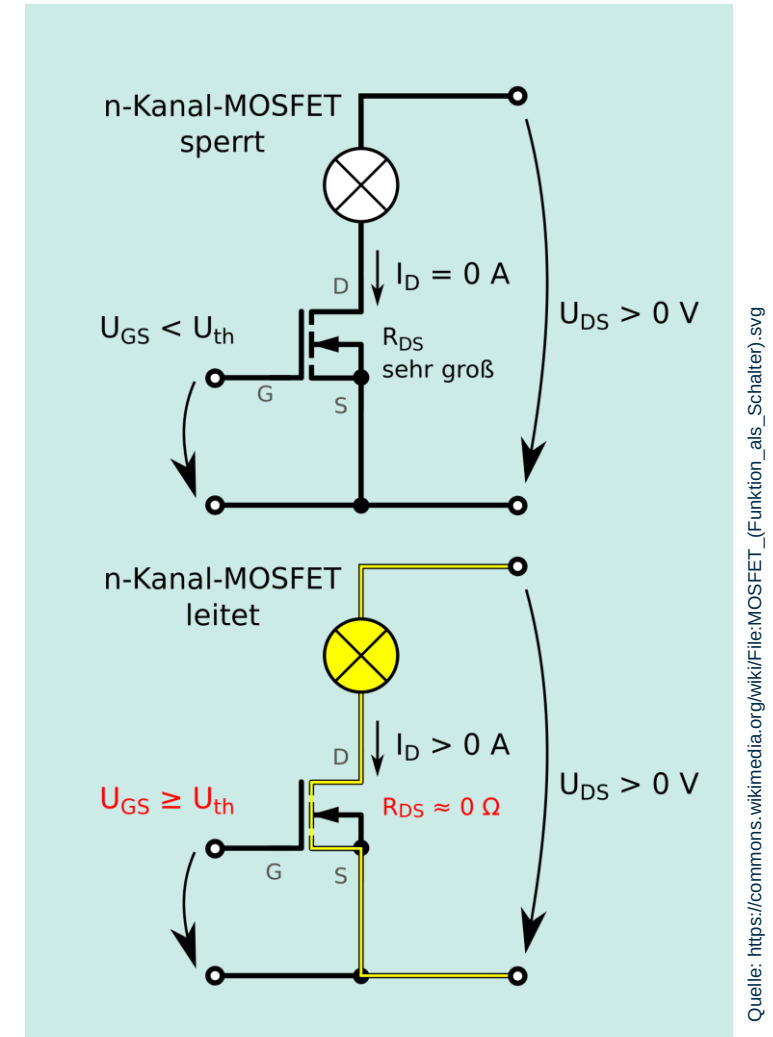
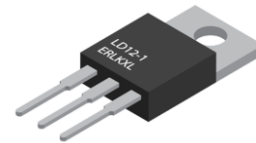
Institut für Technik der Informationsverarbeitung (ITIV)

Abstraktionsebenen



MOSFET

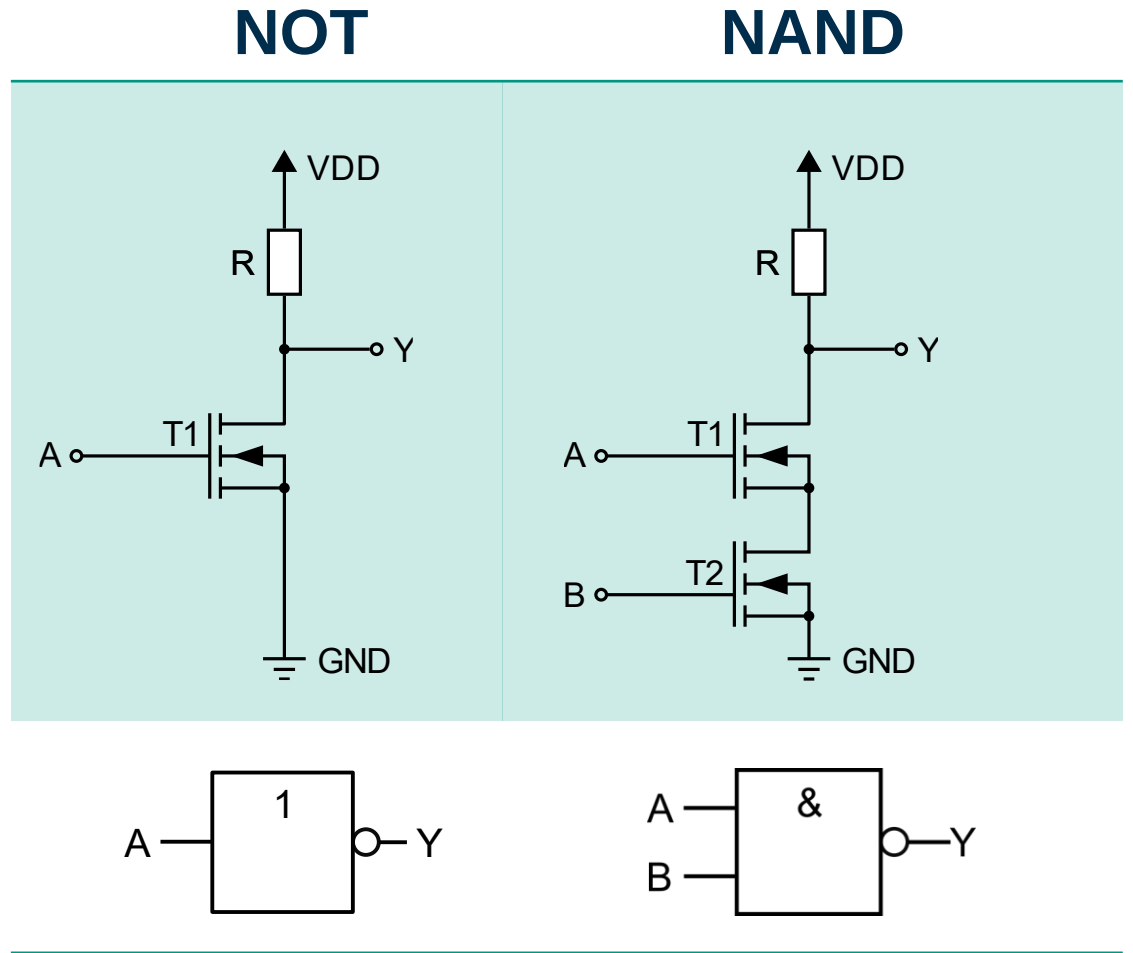
- Metal-Oxide-Semiconductor Field-Effect transistor
- Aktives Bauelement mit mindestens drei Anschlüssen
 - G (gate, Steuerelektrode)
 - D (drain, Abfluss)
 - S (source, Quelle)



Quelle: [https://commons.wikimedia.org/wiki/File:MOSFET_\(Funktion_als_Schalter\).svg](https://commons.wikimedia.org/wiki/File:MOSFET_(Funktion_als_Schalter).svg)

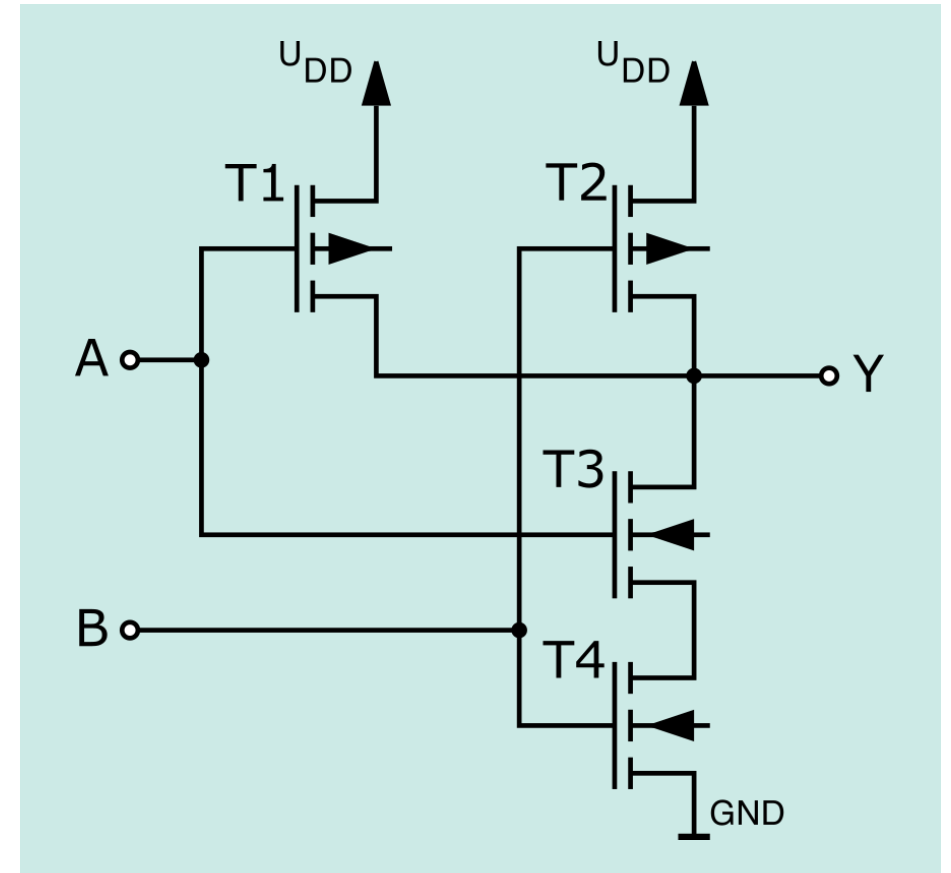
Verschiedene Schaltungen mit MOSFETs

- NMOS-NOT-Gatter aus einem nKanal-Transistor und einem Widerstand
- NMOS-NAND-Gatter aus zwei nKanal-Transistoren und einem Widerstand



CMOS-Technik in der Digitaltechnik

- Complementary Metal-Oxide-Semiconductor
- Kombination von p-Kanal- und n-Kanal-Feldeffekttransistoren
 - p-Kanal-Technik (als Pull-Up-Pfad)
 - n-Kanal-Technik (als Pull-Down-Pfad)
- Strom (von der Versorgungsspannung zur Masse) fließt nur im Umschaltmoment



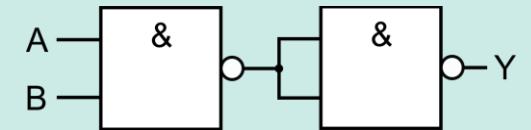
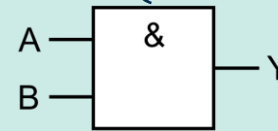
Basissystem der Schaltalgebra

- Normalformtheoreme und der Hauptsatz der Schaltalgebra
 - Jede beliebige Schaltfunktion lässt sich eindeutig durch die drei Grundverknüpfungen Konjunktion, Disjunktion und Negation darzustellen
- NAND-Gattern sind ausreichend um jede beliebige Schaltfunktion zu realisieren

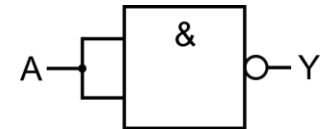
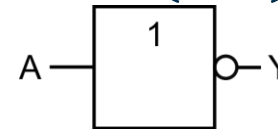
Funktion

Realisierung mit NAND-Gattern

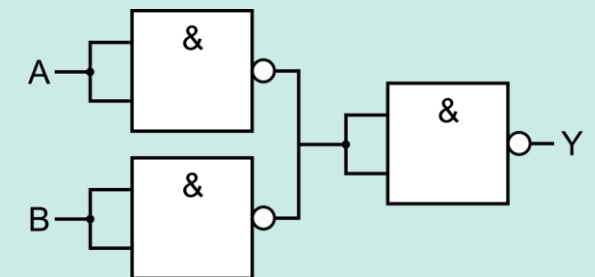
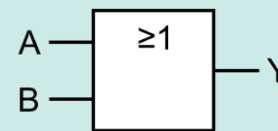
AND: $(A \wedge B)$



NOT: $(\neg A)$

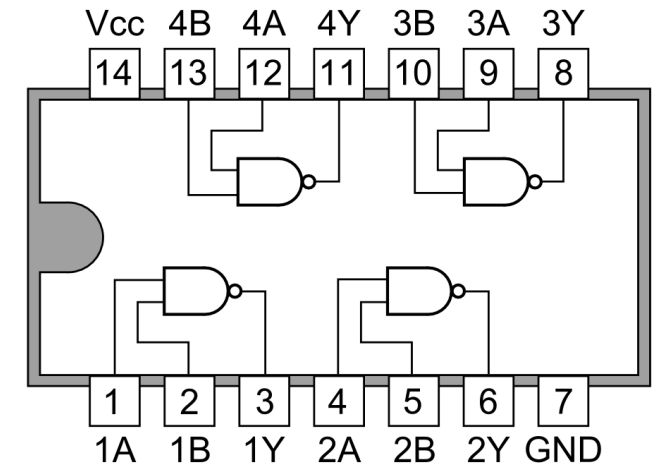
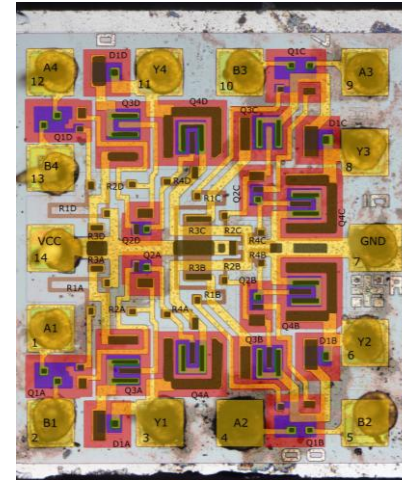


OR: $(A \vee B)$



Logikbaustein 7400

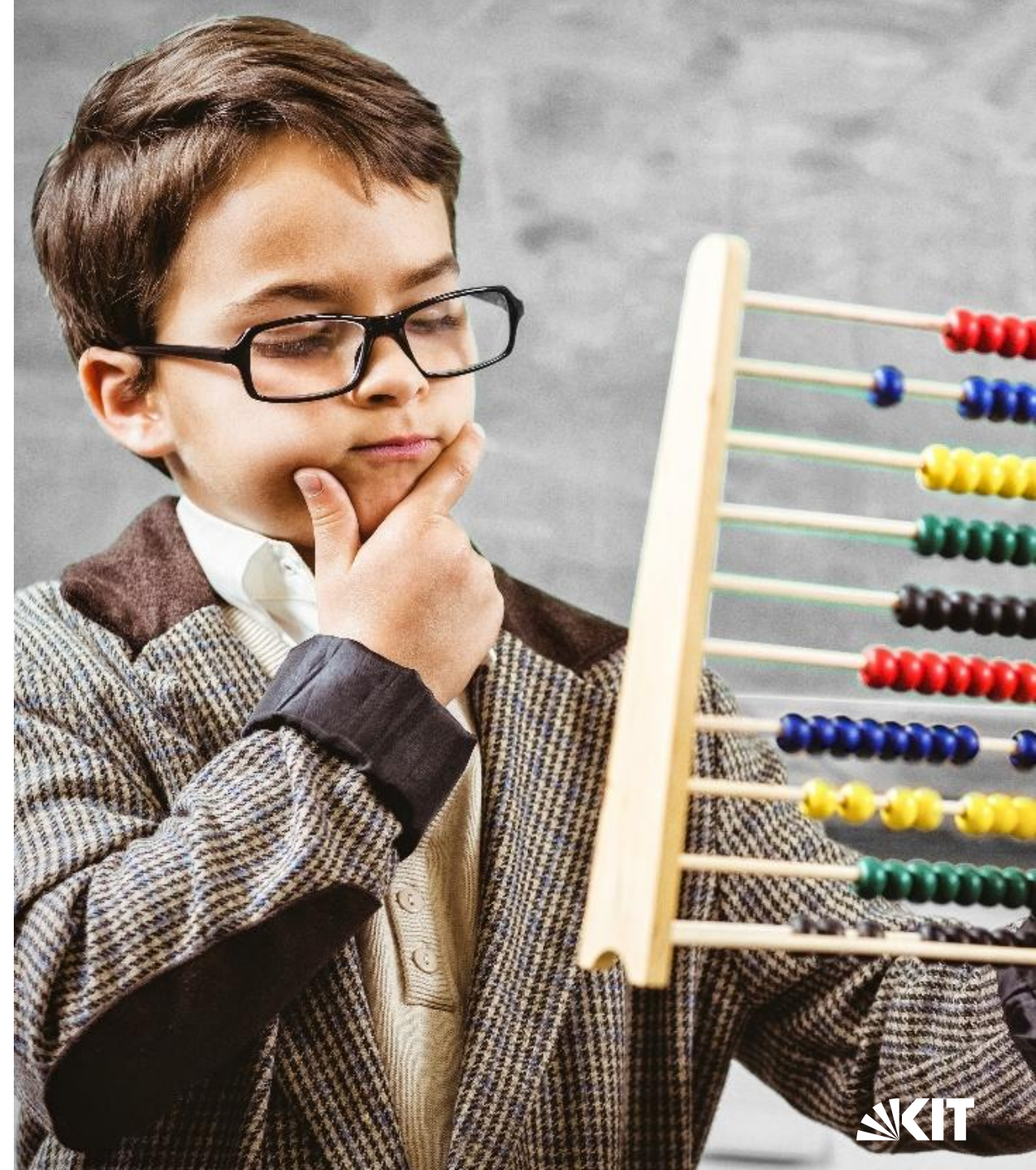
- 4 NAND-Gatter mit jeweils 2 Eingängen
- In den 1960er Jahren entwickelt
- Existieren seitdem funktional praktisch unverändert
- Frühen Heim- und Personal Computer der 1970er und 1980er Jahre enthielten meist eine Vielzahl dieser Chips
- Heutzutage meist durch komplexere ICs ersetzt



Quelle: <https://de.wikipedia.org/wiki/7400>

Von logischen Funktionen zur Arithmetik

- Jede beliebige Schaltfunktion kann realisiert werden
- Können wir mit Hardware auch rechnen?



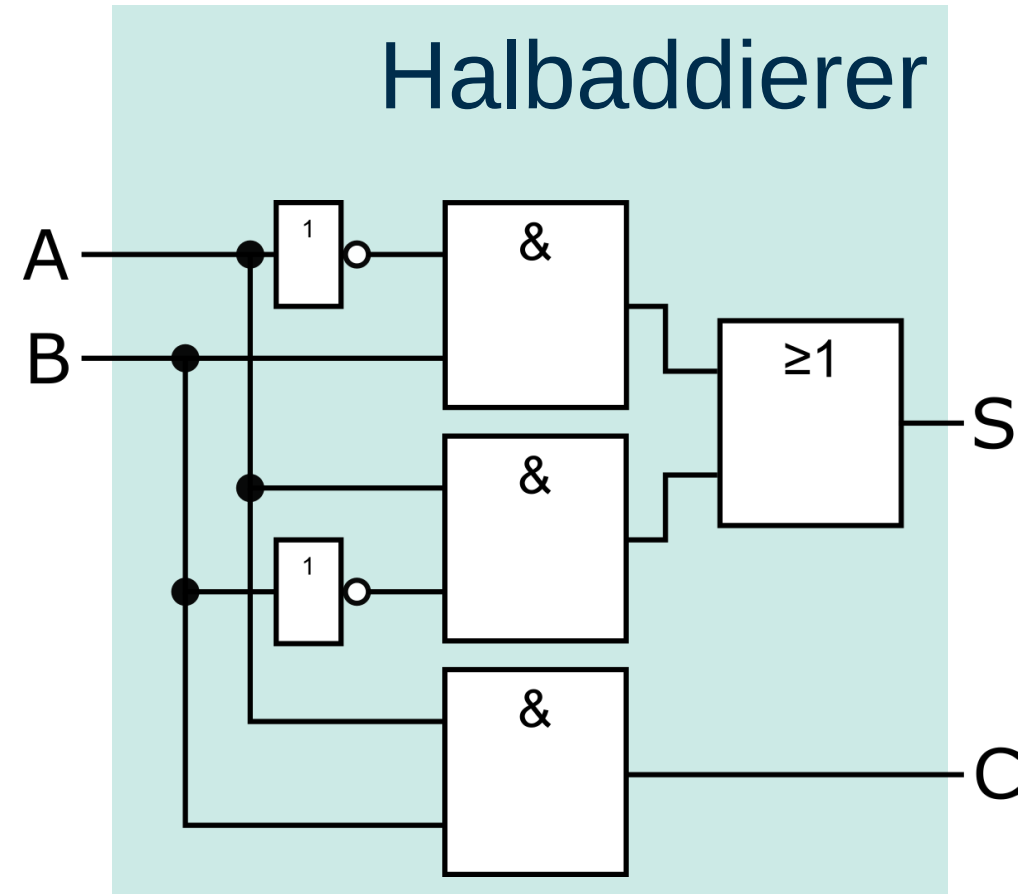
Halbaddierer

- Zwei einstellige Binärzahlen A und B addieren
 - $A + B = CS$
- Summe S
 - $S = (A \wedge \overline{B}) \vee (\overline{A} \wedge B) = A \underline{\vee} B$
- Übertrag C
 - $C = A \wedge B$

A	B	C	S
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

Halbaddierer

- Zwei einstellige Binärzahlen A und B addieren
 - $A + B = CS$
- Summe S
 - $S = (A \wedge \overline{B}) \vee (\overline{A} \wedge B) = A \underline{\vee} B$
- Übertrag C
 - $C = A \wedge B$



Volladdierer

- Drei einstellige Binärzahlen A , B und C addieren
 - $A + B + C_0 = CS$
- Summe S
 - $S = A \underline{\vee} B \underline{\vee} C_0$
- Übertrag C
 - $C = (C_0 \wedge (A \underline{\vee} B)) \vee (A \wedge B)$

A	B	C_0	C	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

Volladdierer

- Drei einstellige Binärzahlen A , B und C addieren

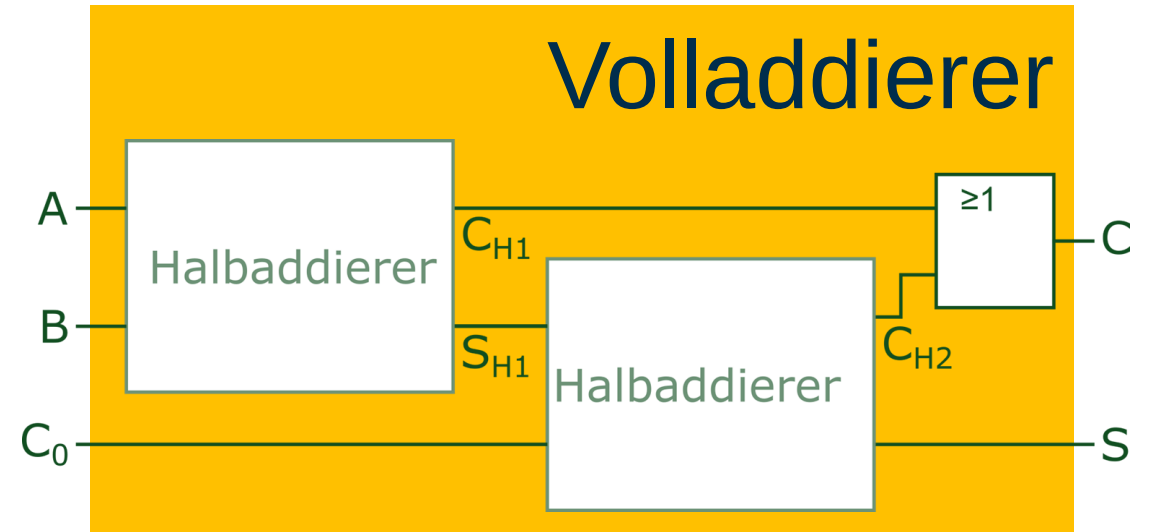
- $A + B + C_0 = CS$

- Summe S

- $S = \underbrace{A \underline{\vee} B}_{S_{H1}} \underline{\vee} C_0$

- Übertrag C

- $C = \underbrace{(C_0 \wedge \underbrace{(A \underline{\vee} B)}_{S_{H1}})}_{C_{H2}} \vee \underbrace{(A \wedge B)}_{C_{H1}}$



Carry-Ripple-Addierer

- Addition von zwei Binärzahlen mit n Stellen

- $A_n \cdots A_2 A_1 A_0 + B_n \cdots B_2 B_1 B_0 = C_{n+1} S_n \cdots S_2 S_1 S_0$

- Summe S_i

- $S_i = A_i \underline{\vee} B_i \underline{\vee} C_i \quad (i = 0, \dots, n - 1)$

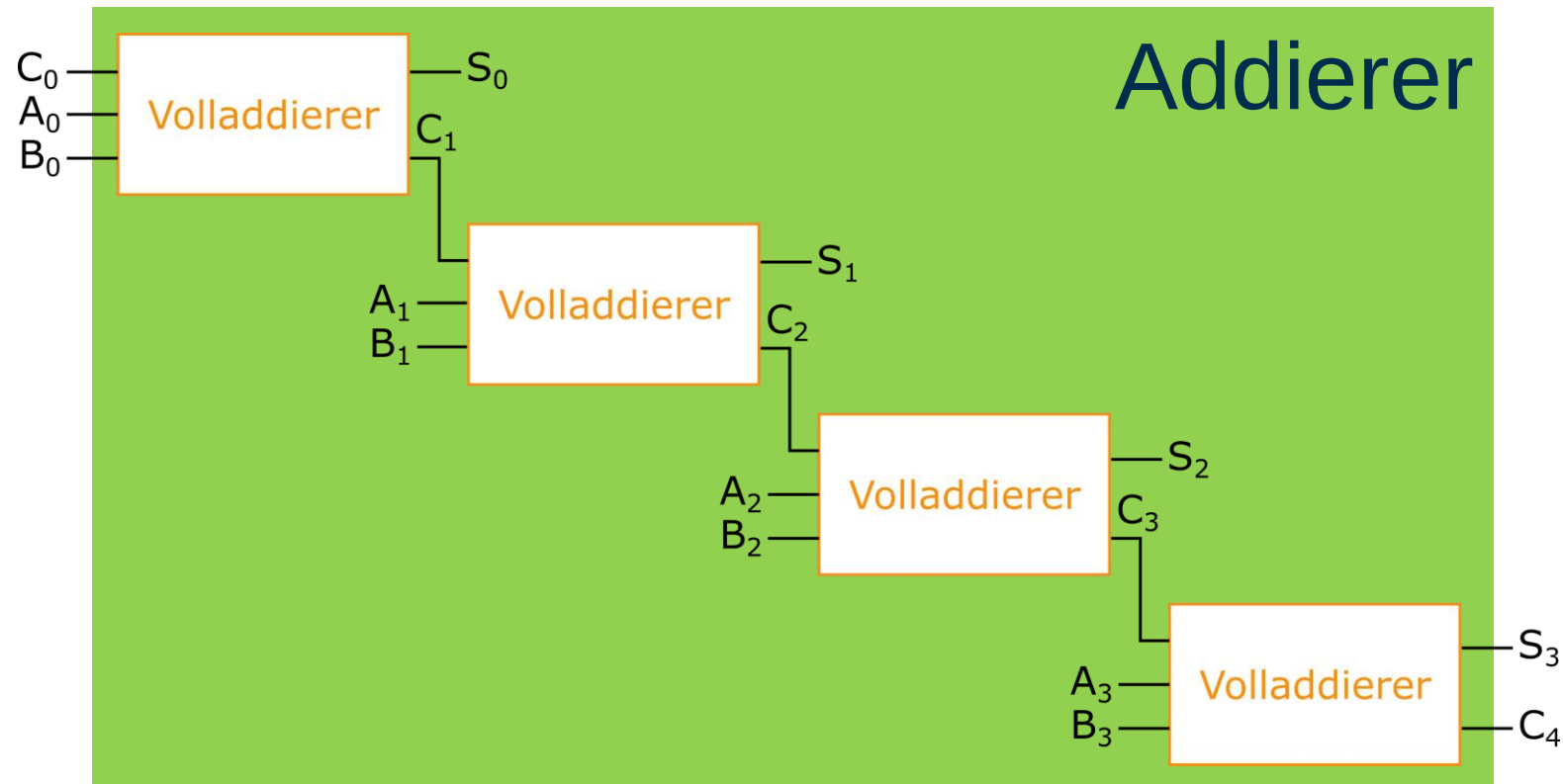
- Übertrag C_{i+1}

- $C_{i+1} = (A_i \wedge B_i) \vee (A_i \wedge C_i) \vee (B_i \wedge C_i)$
 $(i = 0, \dots, n - 1)$

A_i	B_i	C_i	C_{i+1}	S_i
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

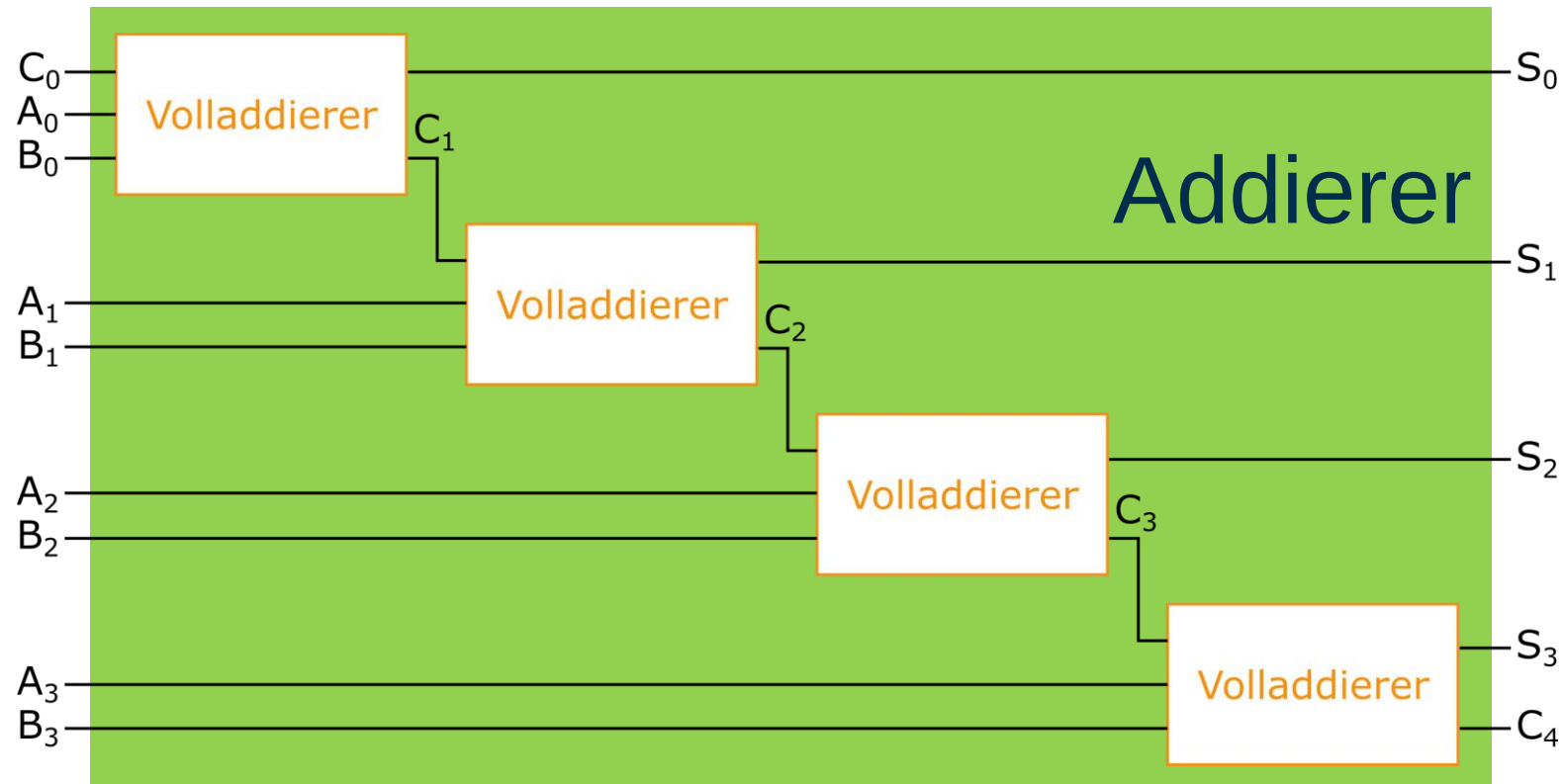
Beispiel Carry-Ripple-Addierer (n=4)

- Addition mehrstelliger Binärzahlen $A_3A_2A_1A_0 + B_3B_2B_1B_0 = C_4S_3S_2S_1S_0$



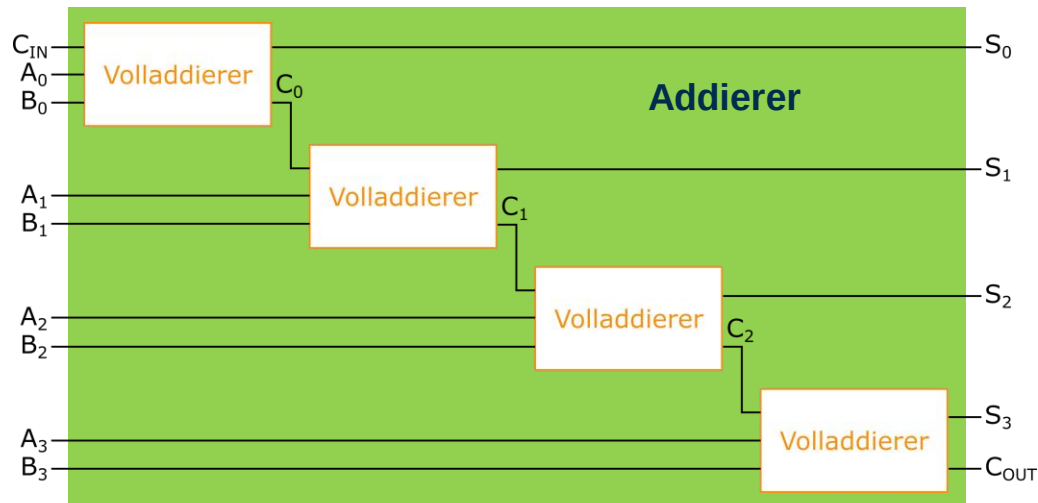
Beispiel Carry-Ripple-Addierer (n=4)

- Addition mehrstelliger Binärzahlen $A_3A_2A_1A_0 + B_3B_2B_1B_0 = C_4S_3S_2S_1S_0$



Carry-Ripple-Addierer

- Addition mehrstelliger Binärzahlen:
 - $A_3A_2A_1A_0 + B_3B_2B_1B_0 = CS_3S_2S_1S_0$
 - hier: $A_n = (A_3, A_2, A_1, A_0)$



Carry-Ripple-Addierer - Alternativen

- **Nachteil: Lange Laufzeit**
 - Verzögerung der Schaltung wächst linear mit Anzahl seiner Eingänge ($O(n)$)
- **Carry-Skip**
 - Volladdierer in Gruppen eingeteilt
 - Schnelle Zusatzlogik für den Übertrag der Gruppe
- **Carry-Look-Ahead (CLA)**
 - Verzögerung der Schaltung nur logarithmisch zur Zahl seiner Eingänge ($O(\log(n))$)
 - Großteil der Berechnungen ohne eingehenden Übertrag vorher berechnen
- **Conditional-Sum-Addition (CSA)**
 - Verzögerung der Schaltung nur logarithmisch zur Zahl seiner Eingänge ($O(\log(n))$)
 - Teile-und-herrsche-Prinzip
- **Carry-Select-Addierer (CSCA)**
 - Verzögerung der Schaltung nur logarithmisch zur Zahl seiner Eingänge ($O(\sqrt{n})$)
 - Parallelisierte Vorabberechnung mit Carry-Ripple-Addierern mit und ohne Übertrag

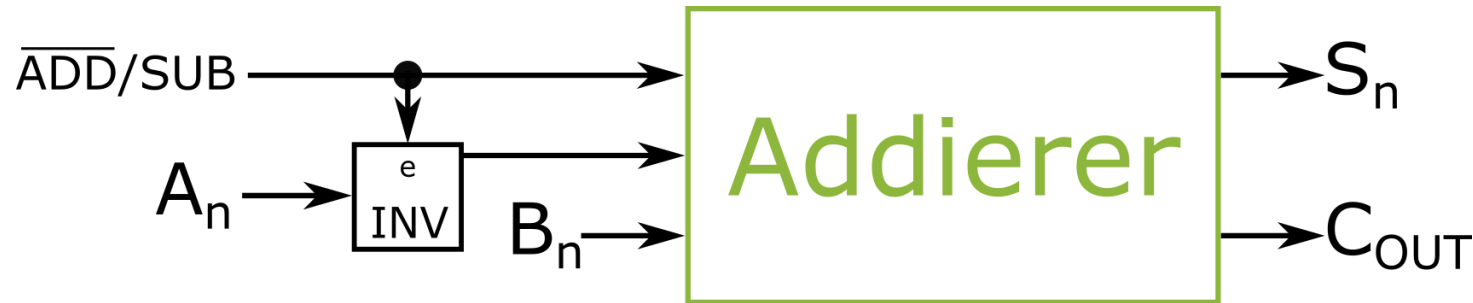
Von Addition zu Subtraktion

- Subtraktion mehrstelliger Binärzahlen: $B_3B_2B_1B_0 - A_3A_2A_1A_0 = S_3S_2S_1S_0$
- Zweierkompliment nutzen



Von Addition zu Subtraktion

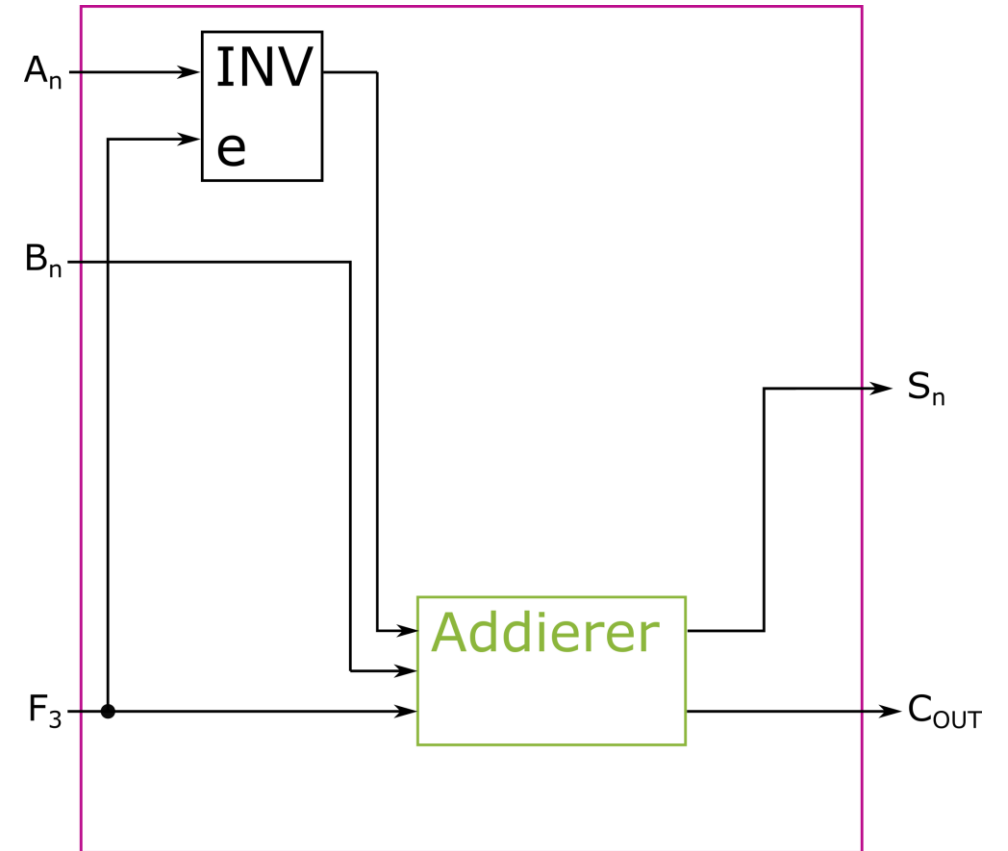
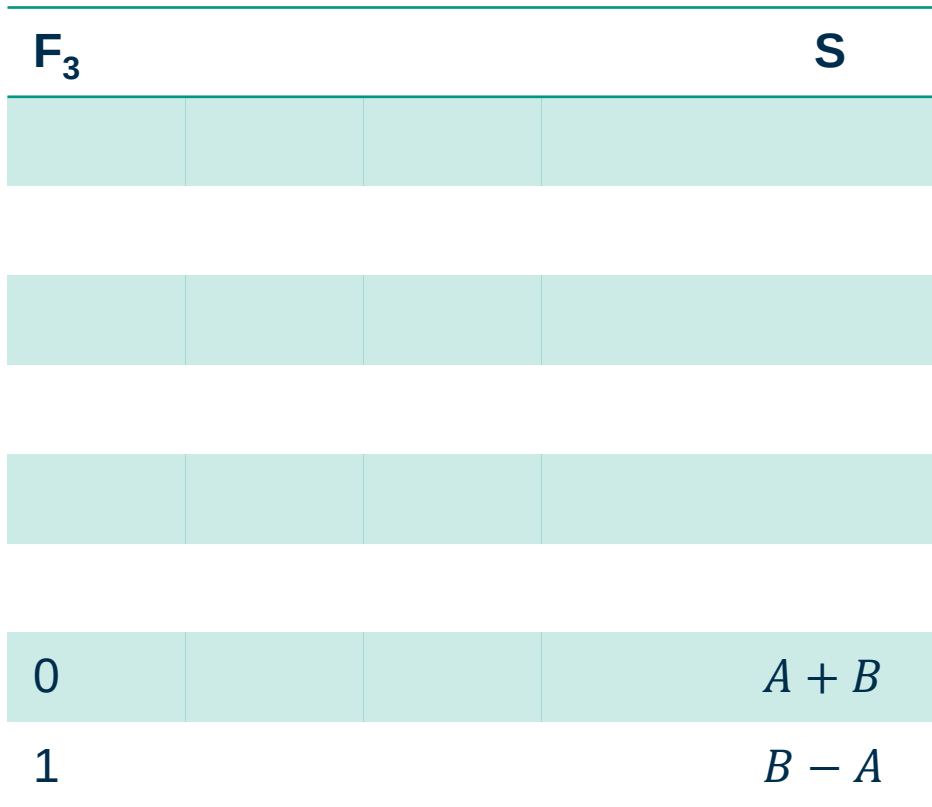
- Addition mehrstelliger Binärzahlen: $A_3A_2A_1A_0 + B_3B_2B_1B_0 = C_4S_3S_2S_1S_0$
- Subtraktion mehrstelliger Binärzahlen: $B_3B_2B_1B_0 - A_3A_2A_1A_0 = S_3S_2S_1S_0$
- Ansteuerung mit einem Kontrollbit $\overline{ADD}/SUB$



Arithmetische Einheit

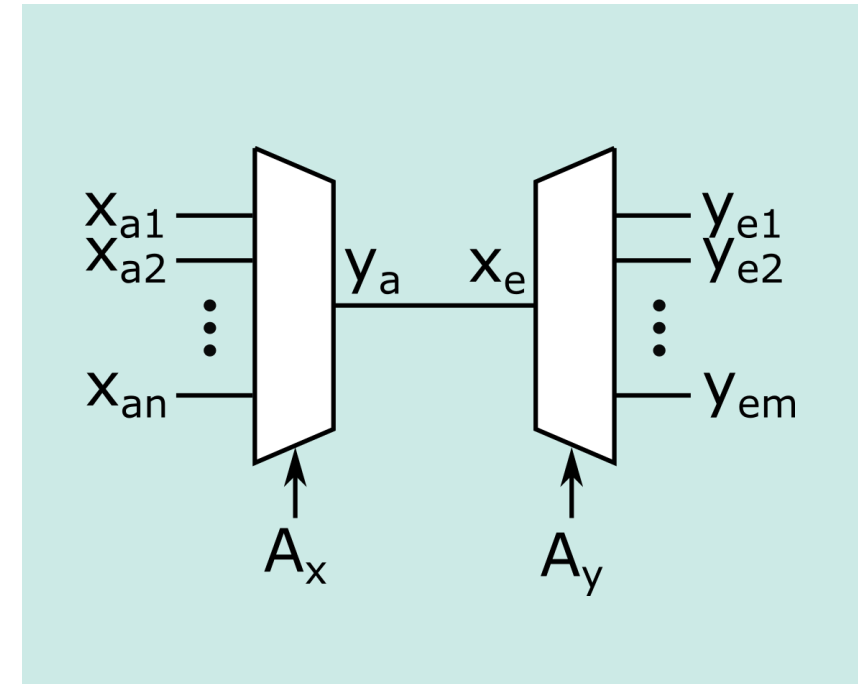
- Arithmetische Operationen

- Addition und Subtraktion



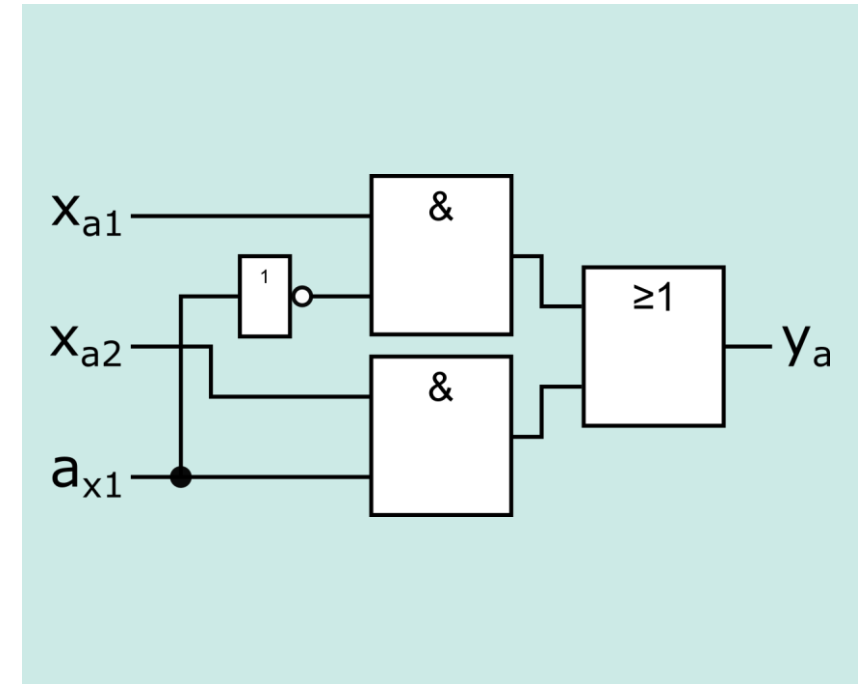
Multiplexer und Demultiplexer

- Zur Selektion von Signalkomponenten und (Wieder-) Zuweisung
- Steuergrößen A_x und A_y selektieren den einzelnen Eingang bzw. den einzelnen Ausgang
- Multiplexer
 - $A_x = (a_{xr}, \dots, a_{x2}, a_{x3})$ und $2^r \geq n$
- Demultiplexer
 - $A_y = (a_{yt}, \dots, a_{y2}, a_{y3})$ und $2^t \geq m$



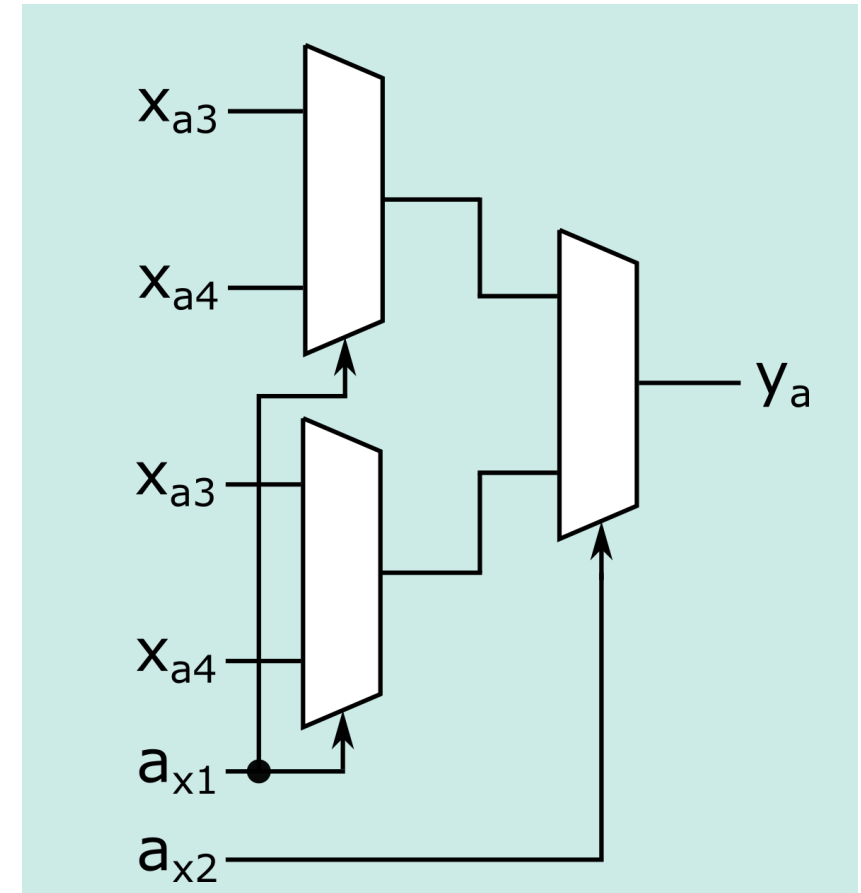
Beispiel 1-MUX

a_{x1}	x_{a1}	x_{a2}	y_a
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	0
1	1	1	1



Beispiel 2-MUX

- Kann aus drei 1-MUX erzeugt werden

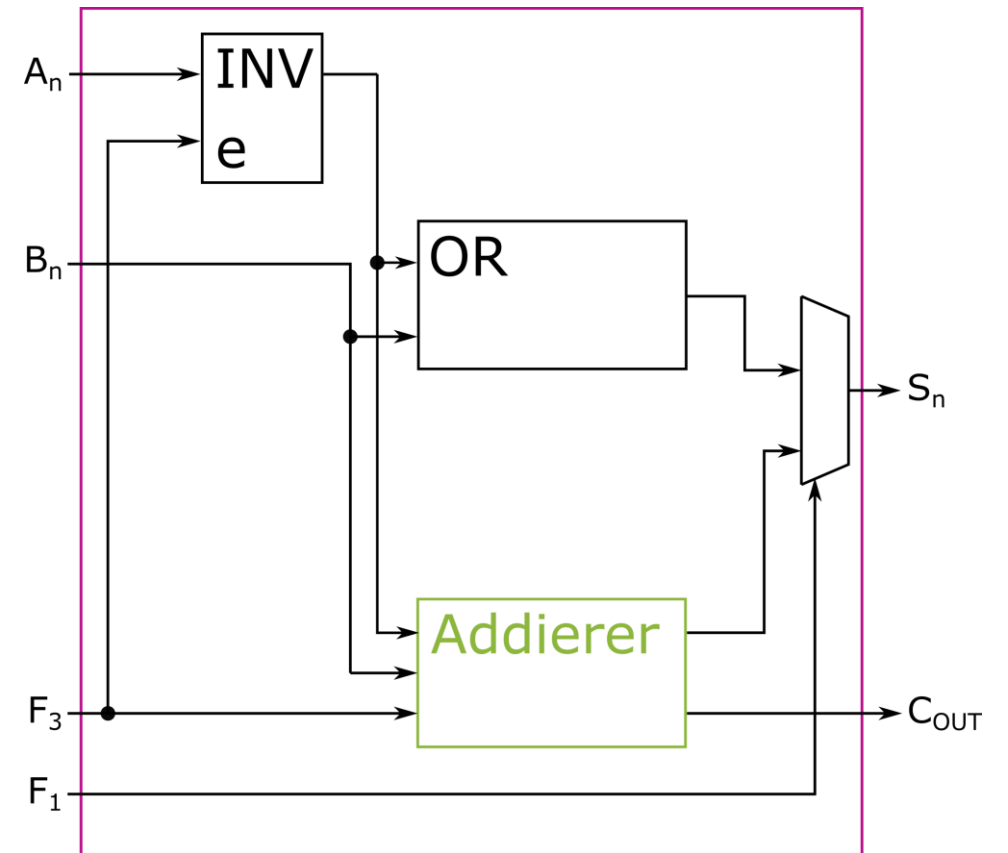


Arithmetisch-logische Einheit

■ Logische Operationen

■ $A \vee B$

F_3	F_1	S
0	0	$A \vee B$
0	1	$A + B$
1	1	$B - A$

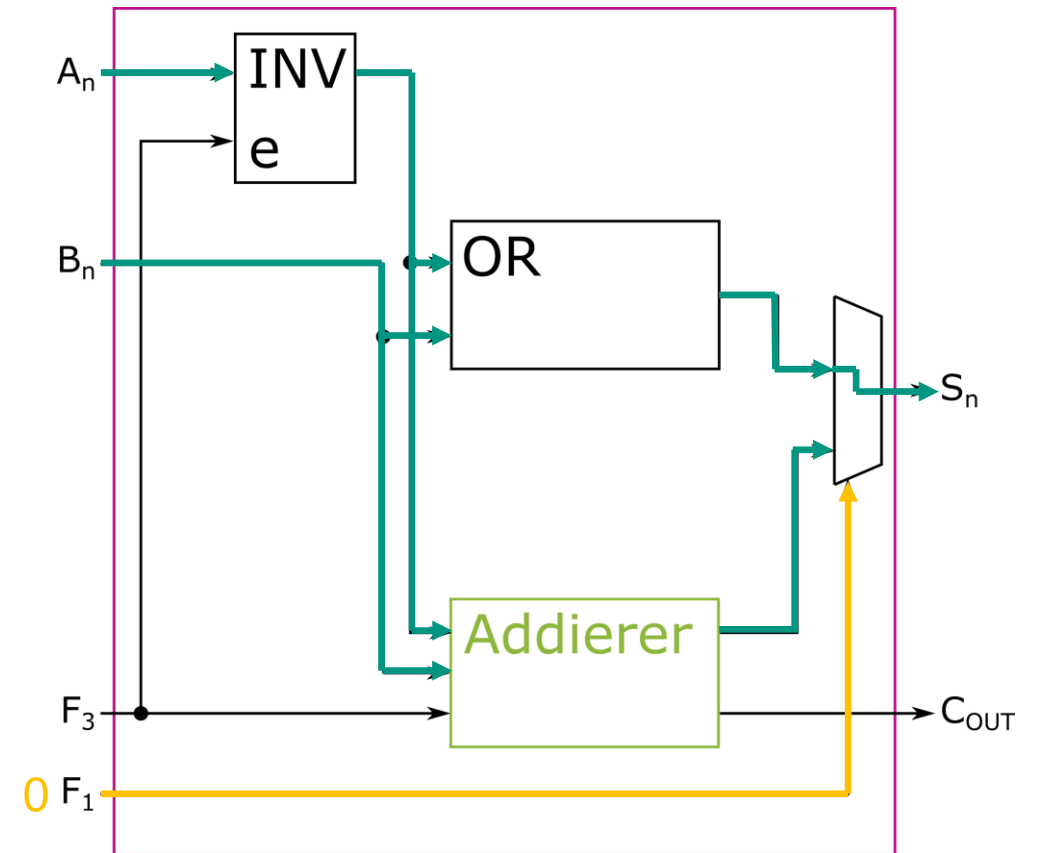


Arithmetisch-logische Einheit

■ Logische Operationen

■ $A \vee B$

F_3		F_1	S
0		0	$A \vee B$
0		1	$A + B$
1		1	$B - A$

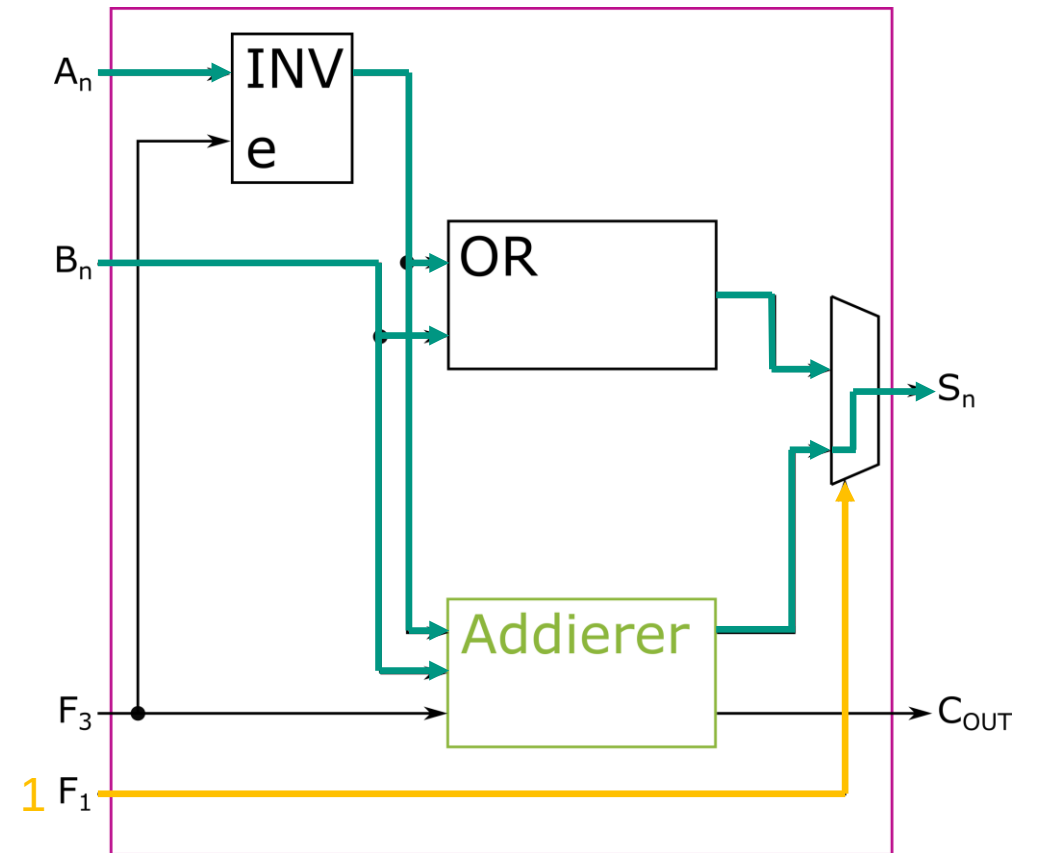


Arithmetisch-logische Einheit

■ Logische Operationen

■ $A \vee B$

F_3		F_1	S
0		0	$A \vee B$
0		1	$A + B$
1		1	$B - A$

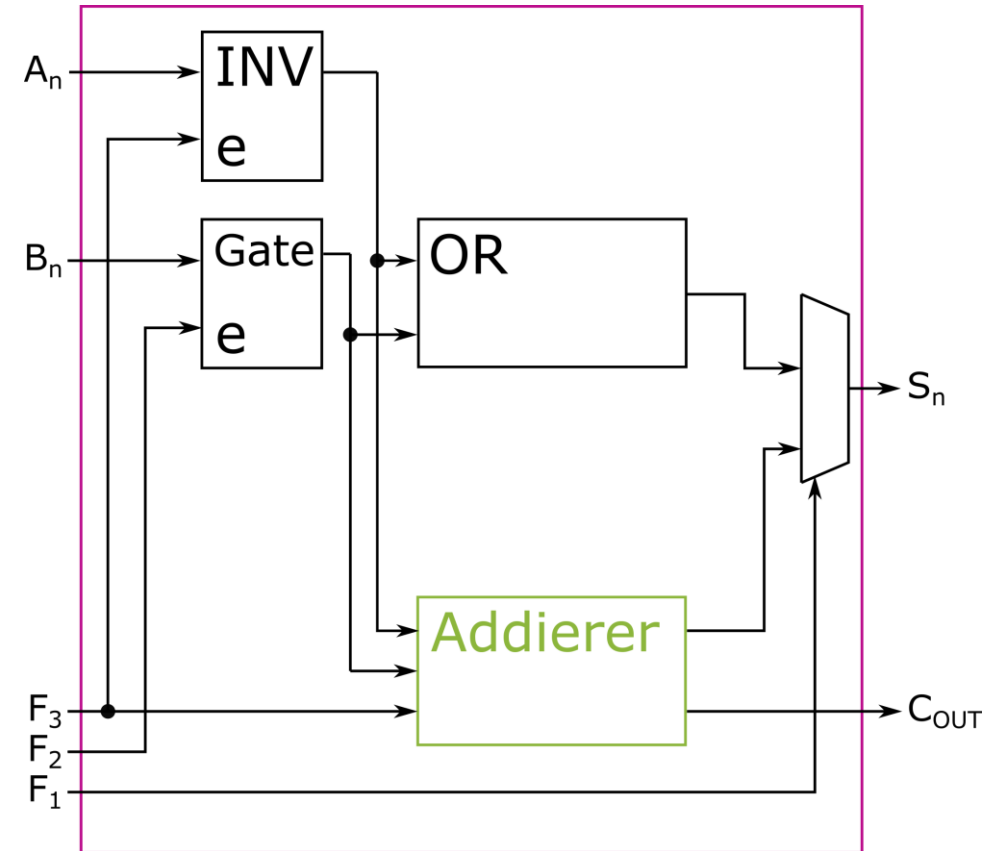


Arithmetisch-logische Einheit

■ Logische Operationen

■ $NOT A$

F_3	F_2	F_1	S
0	0	0	A
1	0	0	$NOT A$
0	1	0	$A \vee B$
0	1	1	$A + B$
1	1	1	$B - A$

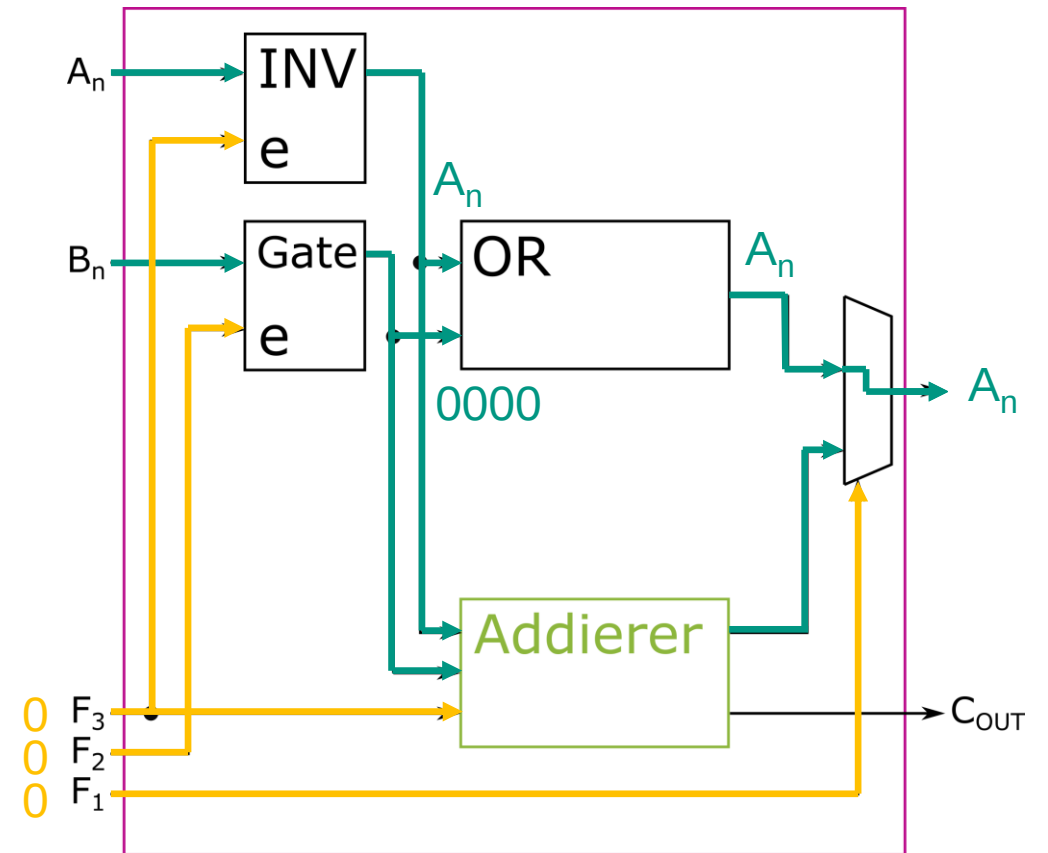


Arithmetisch-logische Einheit

Logische Operationen

NOT A

F ₃	F ₂	F ₁	S
0	0	0	A
1	0	0	NOT A
0	1	0	A ∨ B
0	1	1	A + B
1	1	1	B - A

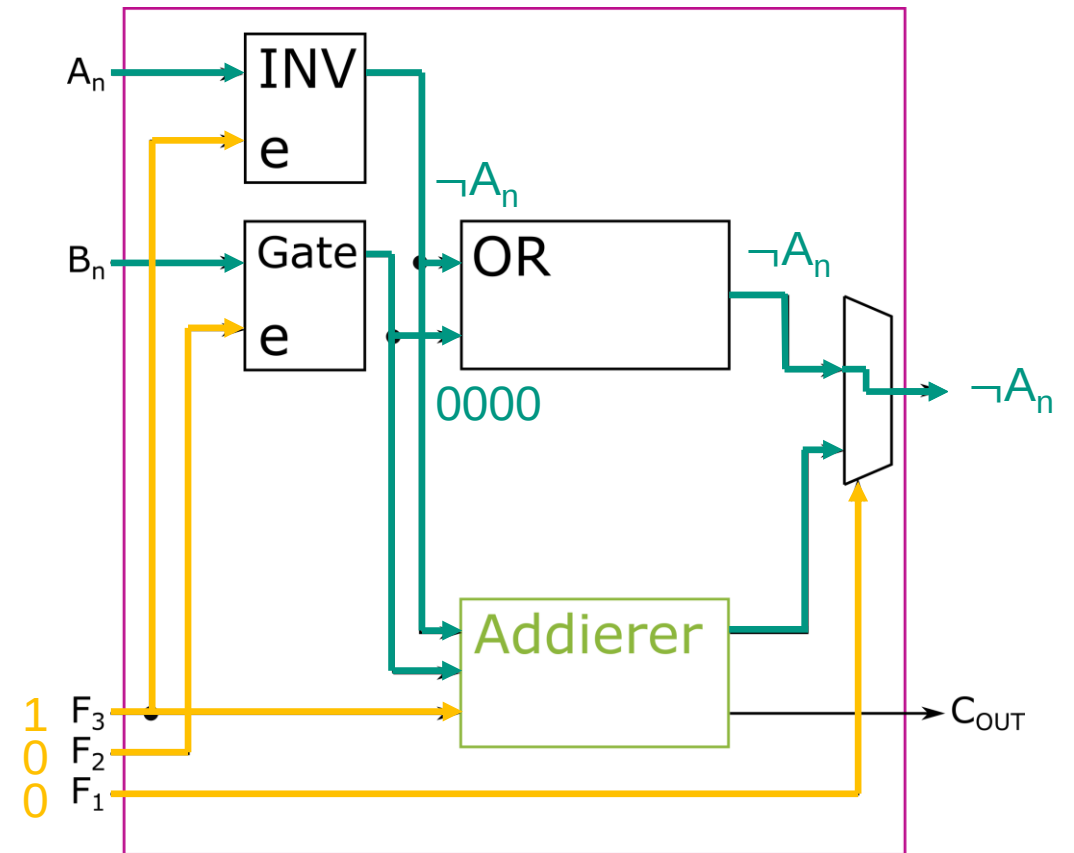


Arithmetisch-logische Einheit

Logische Operationen

NOT A

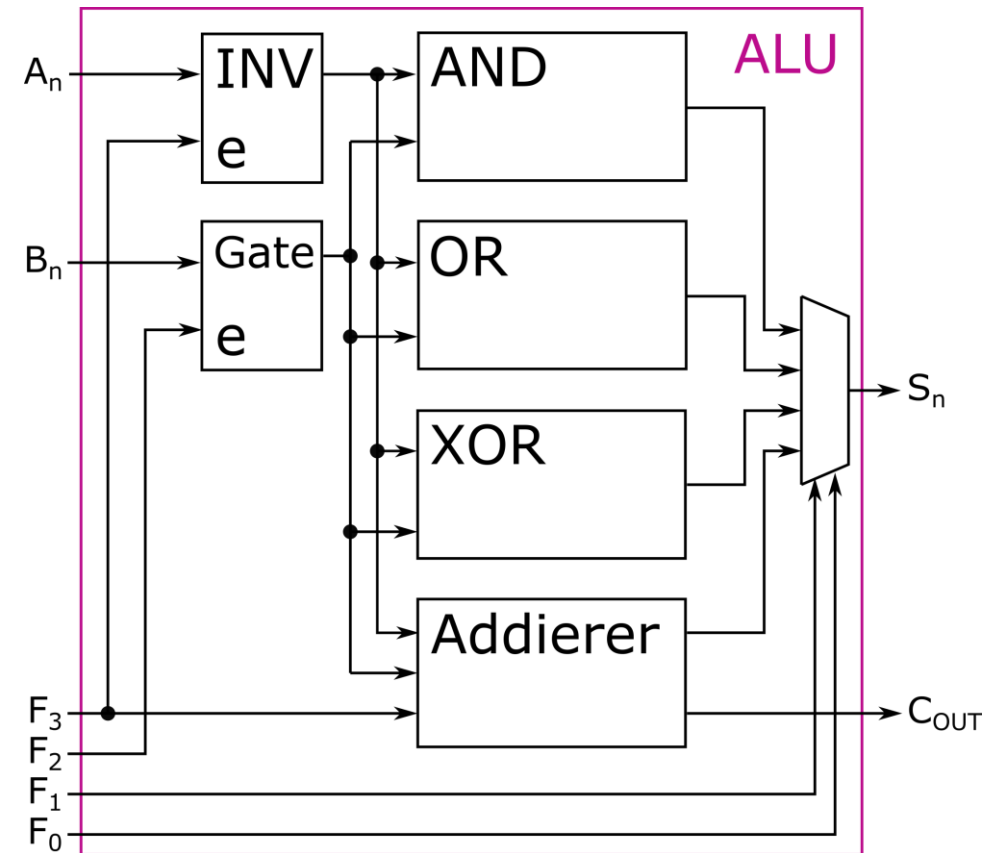
F ₃	F ₂	F ₁	S
0	0	0	A
1	0	0	NOT A
0	1	0	A ∨ B
0	1	1	A + B
1	1	1	B - A



Arithmetisch-logische Einheit

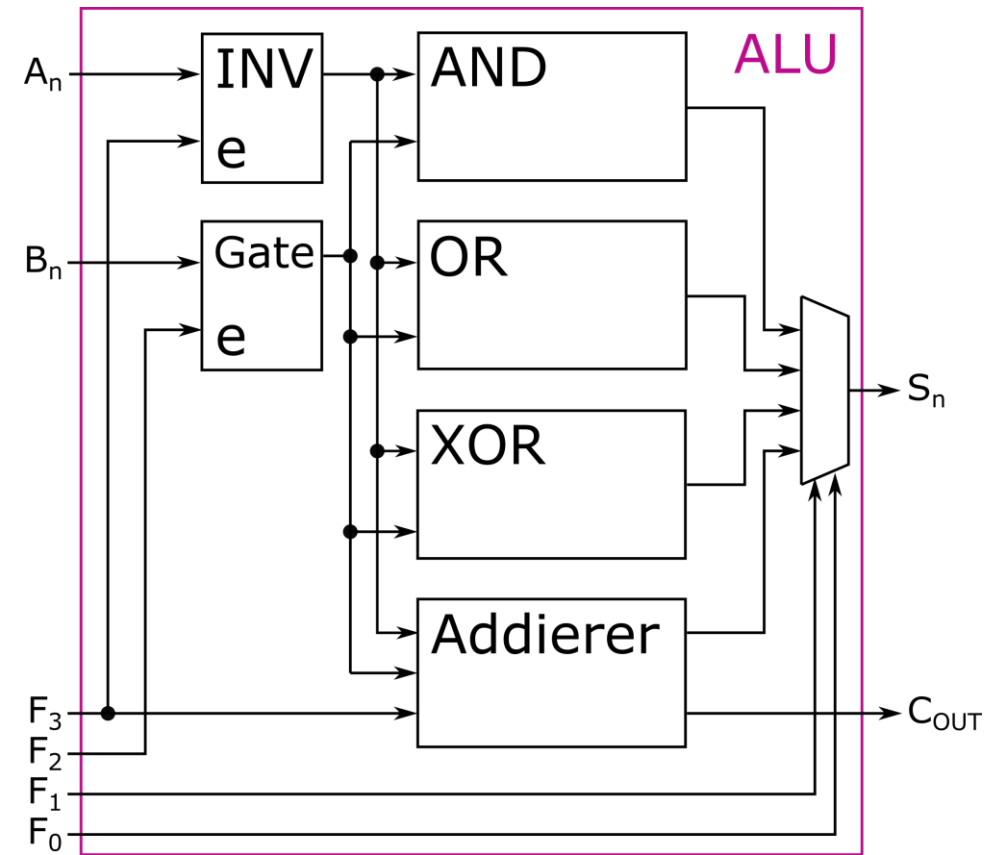
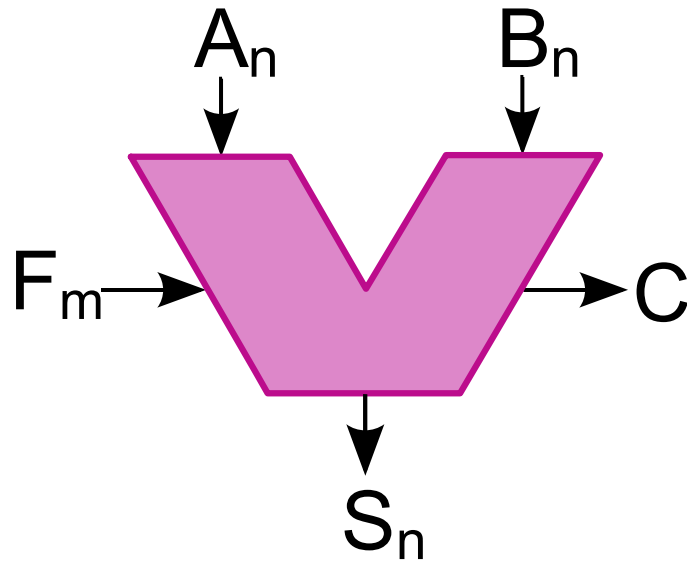
- Eine ALU kann zwei Binärwerte miteinander verknüpfen

F_3	F_2	F_1	F_0	S
0	0	0	0	0
0	0	0	1	A
1	0	0	1	$NOT A$
0	1	0	0	$A \wedge B$
0	1	0	1	$A \vee B$
0	1	1	0	$A \underline{\vee} B$
0	1	1	1	$A + B$
1	1	1	1	$B - A$



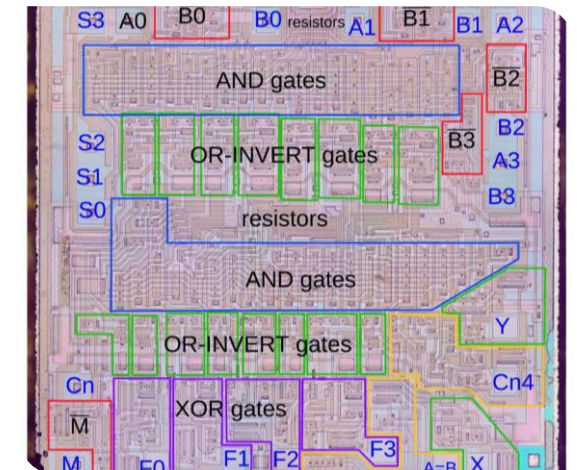
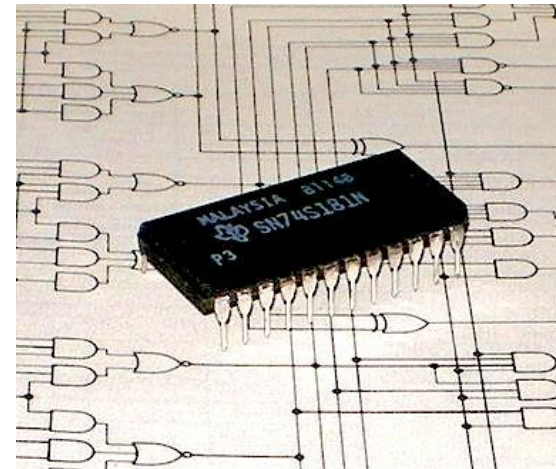
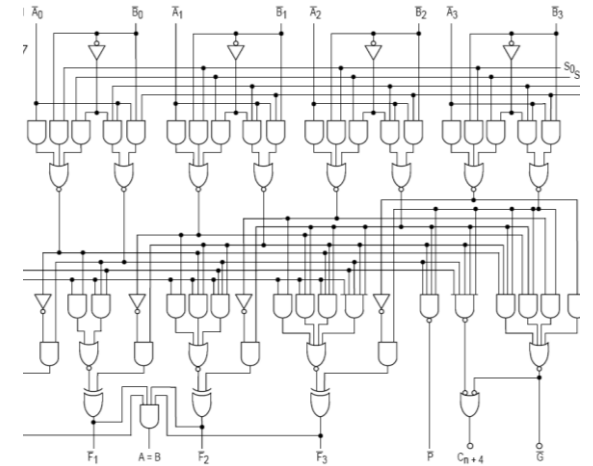
Arithmetisch-logische Einheit

- Eine ALU (arithmetic logic unit) kann zwei Binärwerte mit gleicher Stellenzahl (n) miteinander verknüpfen



Beispiel einer 4-Bit-ALU: 74181

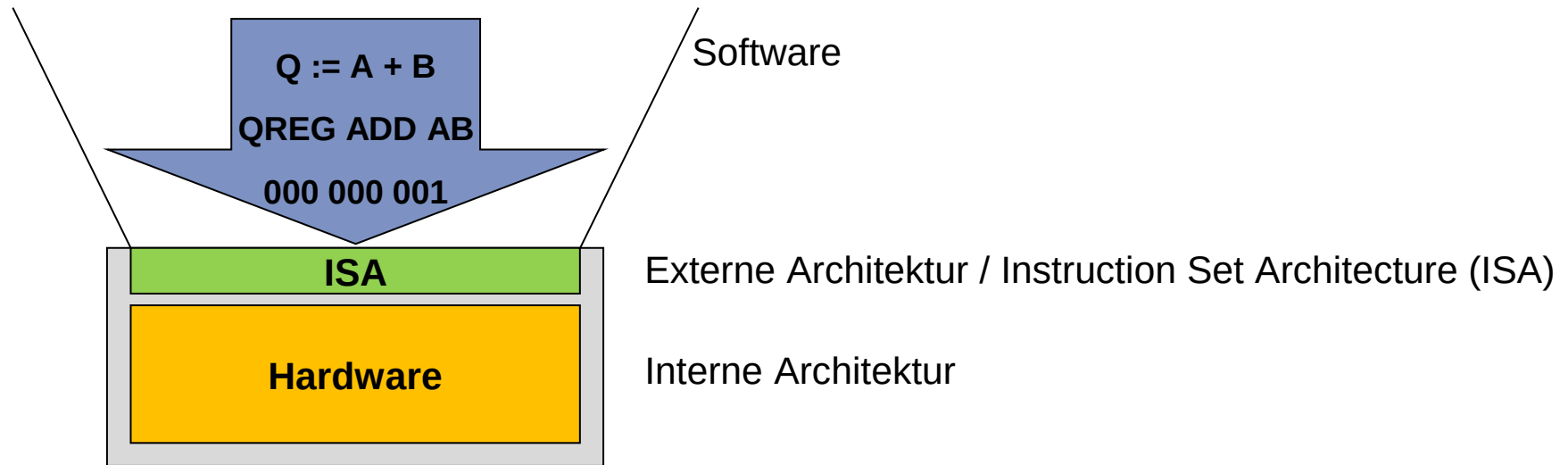
- Erste ALU auf einem Chip (1970)
- Diese ALU kann mit vier Bits je Operand arbeiten
 - alle 16 logischen Operationen mit zwei Variablen
 - Addition mit und ohne Übertrag
 - Subtraktion mit und ohne Übertrag
 - Inkrement und Dekrement
 - Vergleich von Werten auf gleich, größer oder kleiner
- <200 Transistoren



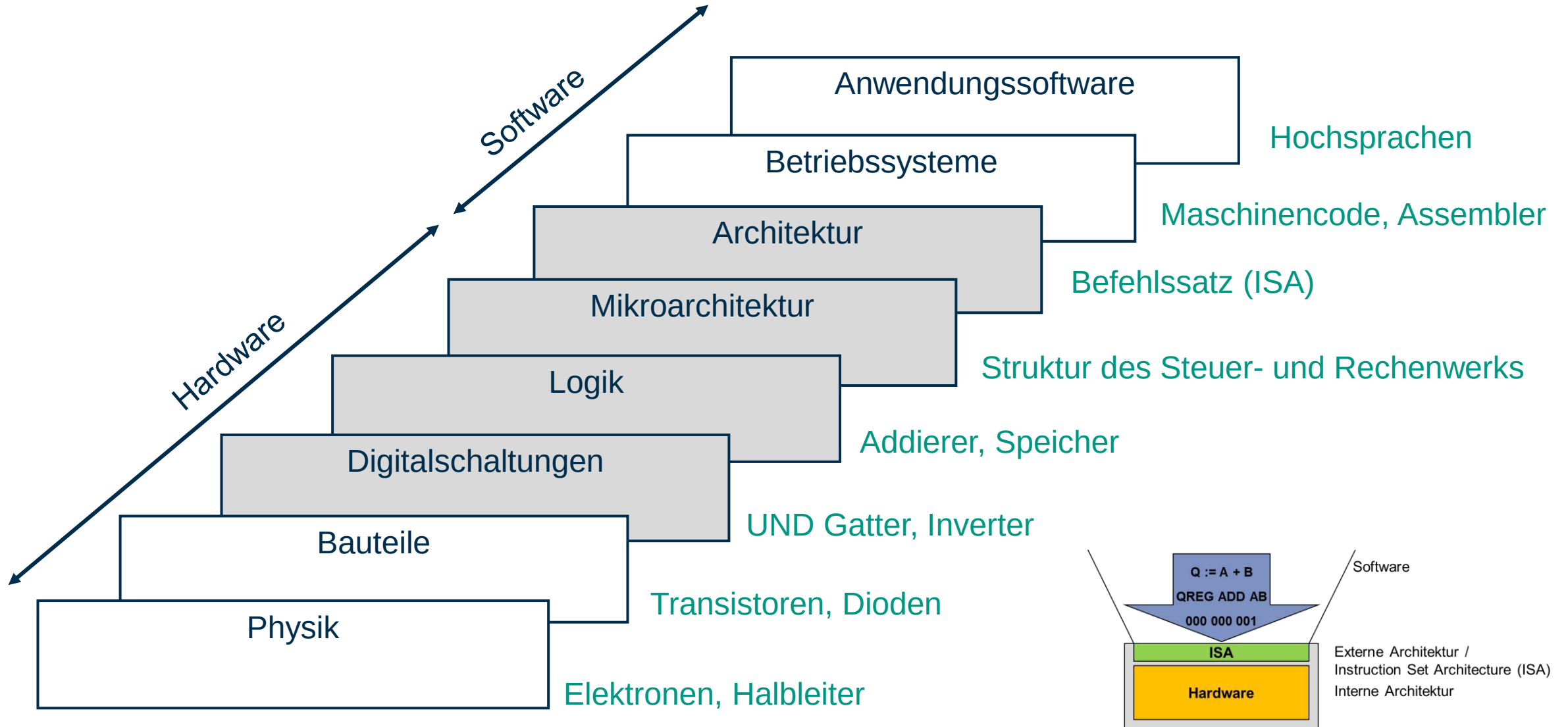
Ansteuerung einer ALU

- Wie kommen Daten in die ALU?
- Wie werden die einzelnen Operationen gesteuert?

➡ Maschinencode

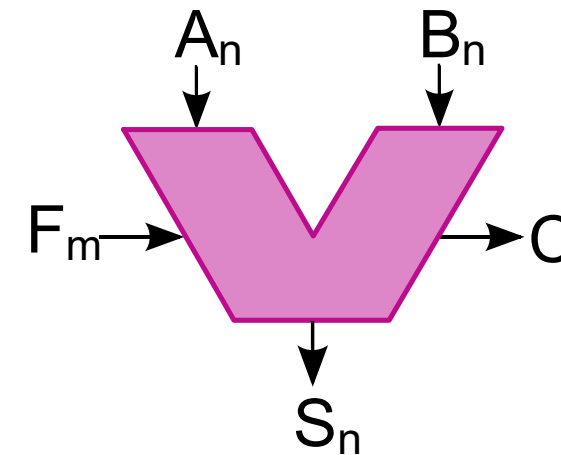
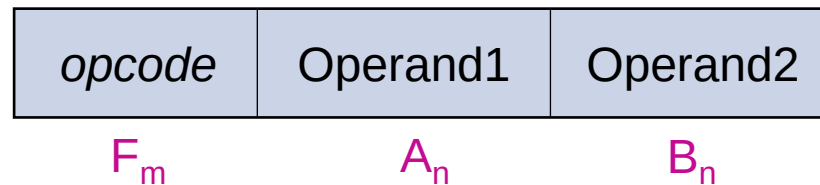


Abstraktionsebenen



Beispiel ALU - Maschinencode

- Vier Bits für die Codierung, was getan werden soll (F0-F3)
 - Wird opcode genannt
- Zahlen, die für die Berechnung benötigt werden
 - Werden Operanden genannt
- Eine Instruktion/Maschinenbefehl sieht dann beispielsweise so aus:



Beispiel ALU - Maschinencode

- Addition $6 + 5$
 - opcode für $(A + B)$ ist 0111
 - $A = 0110$ und $B = 0101$

<i>opcode</i>	Operand1	Operand2
---------------	----------	----------

F_3	F_2	F_1	F_0	S
0	0	0	0	0
0	0	0	1	A
1	0	0	1	$NOT A$
0	1	0	0	$A \wedge B$
0	1	0	1	$A \vee B$
0	1	1	0	$A \underline{\vee} B$
0	1	1	1	$A + B$
1	1	1	1	$B - A$

Beispiel ALU - Maschinencode

- Addition $6 + 5$
 - opcode für $(A + B)$ ist 0111
 - $A = 0110$ und $B = 0101$

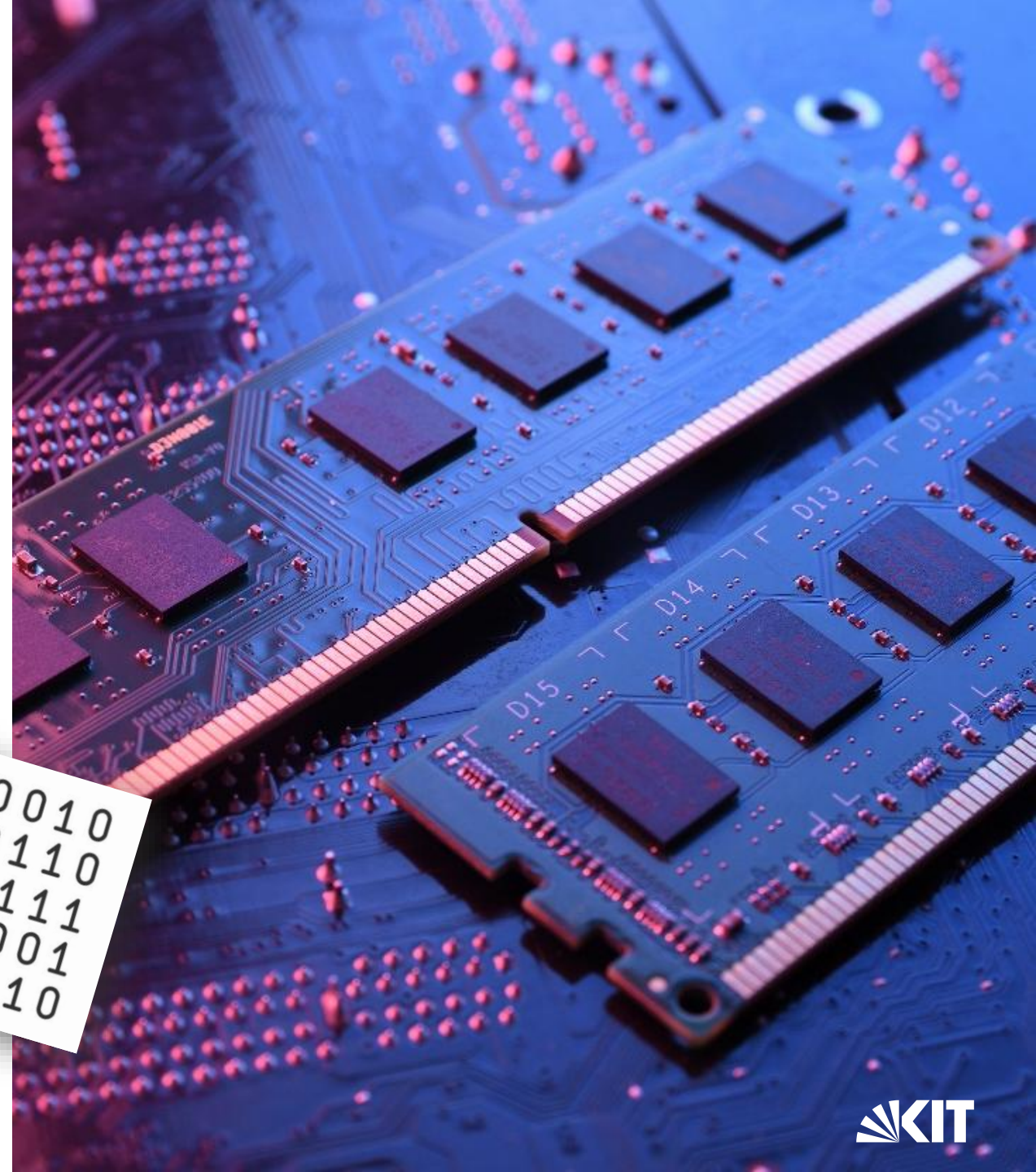
0111	0110	0101
------	------	------

- Im Speicher steht 011101100101

F_3	F_2	F_1	F_0	S
0	0	0	0	0
0	0	0	1	A
1	0	0	1	$NOT A$
0	1	0	0	$A \wedge B$
0	1	0	1	$A \vee B$
0	1	1	0	$A \underline{\vee} B$
0	1	1	1	$A + B$
1	1	1	1	$B - A$

Was ist ein Datenspeicher?

- Elektronisches Bauteil, das Daten speichert
- Daten werden in Form von binären Zuständen gespeichert
- Es gibt Adressen für Speicherbereiche
 - Daten können mit ihrer Adresse gefunden werden

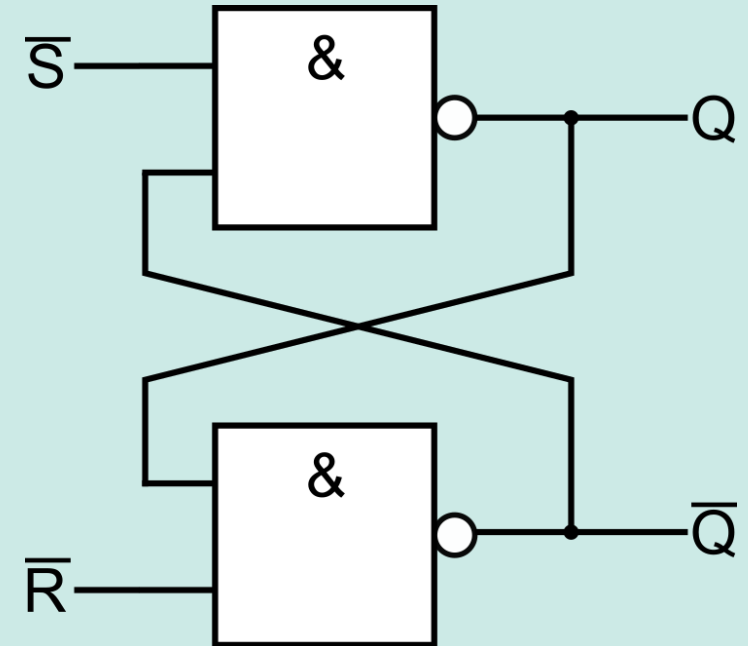


Ungetaktetes RS – FlipFlop

- Charakteristische Gleichung

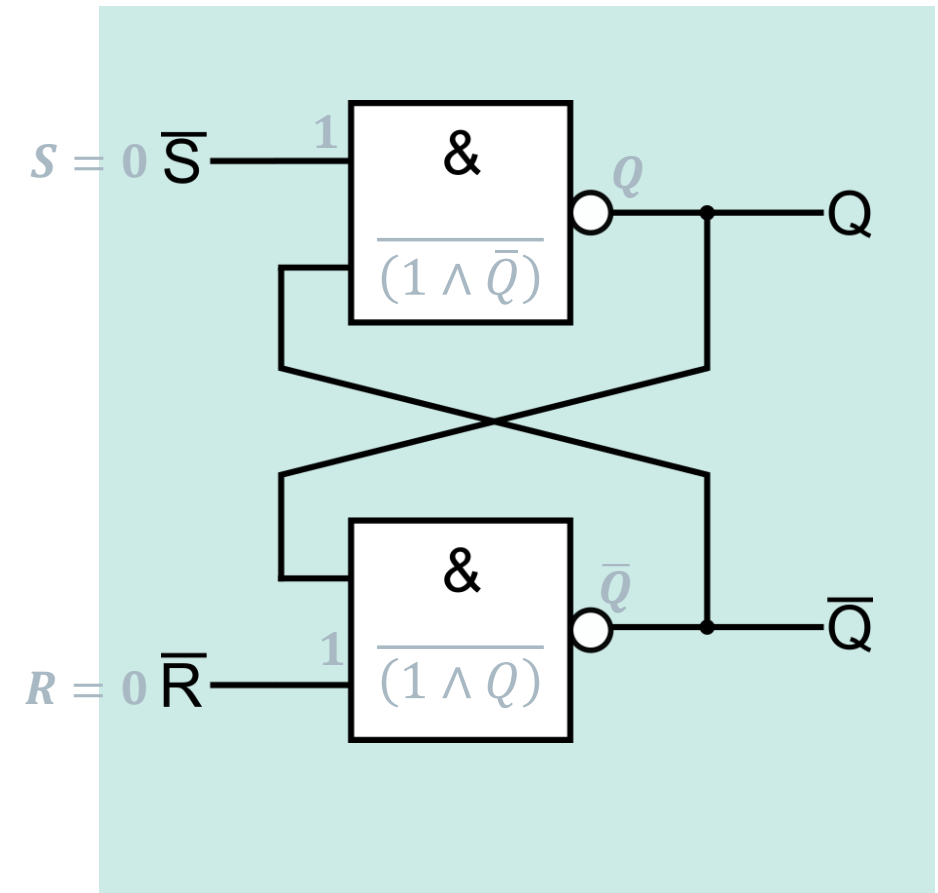
- $$\begin{aligned} Q &= \overline{(\bar{S} \wedge \bar{Q})} \\ &= S \vee \bar{Q} \\ &= S \vee \overline{(Q \wedge \bar{R})} \\ &= S \vee (Q \wedge \bar{R}) \end{aligned}$$

- Ausgangssignal ist abhängig von seiner Vorgeschichte



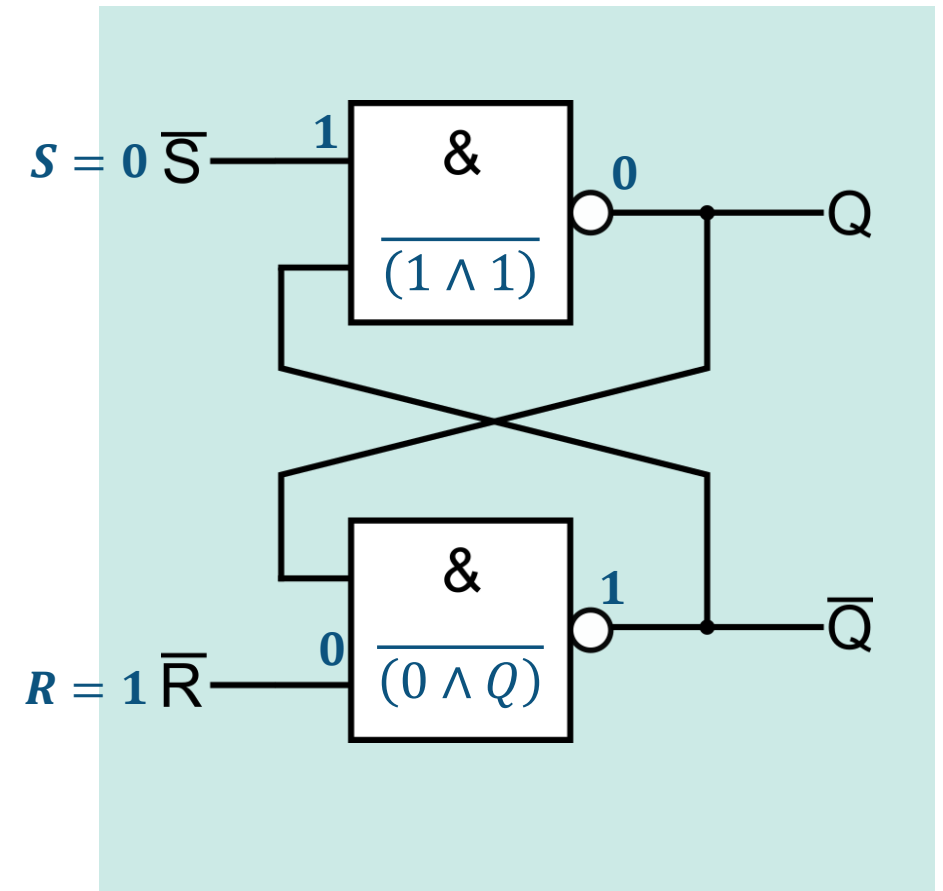
Ungetaktetes RS – FlipFlop

S	R	Q
0	0	Q
0	1	0
1	0	1
1	1	- (1)



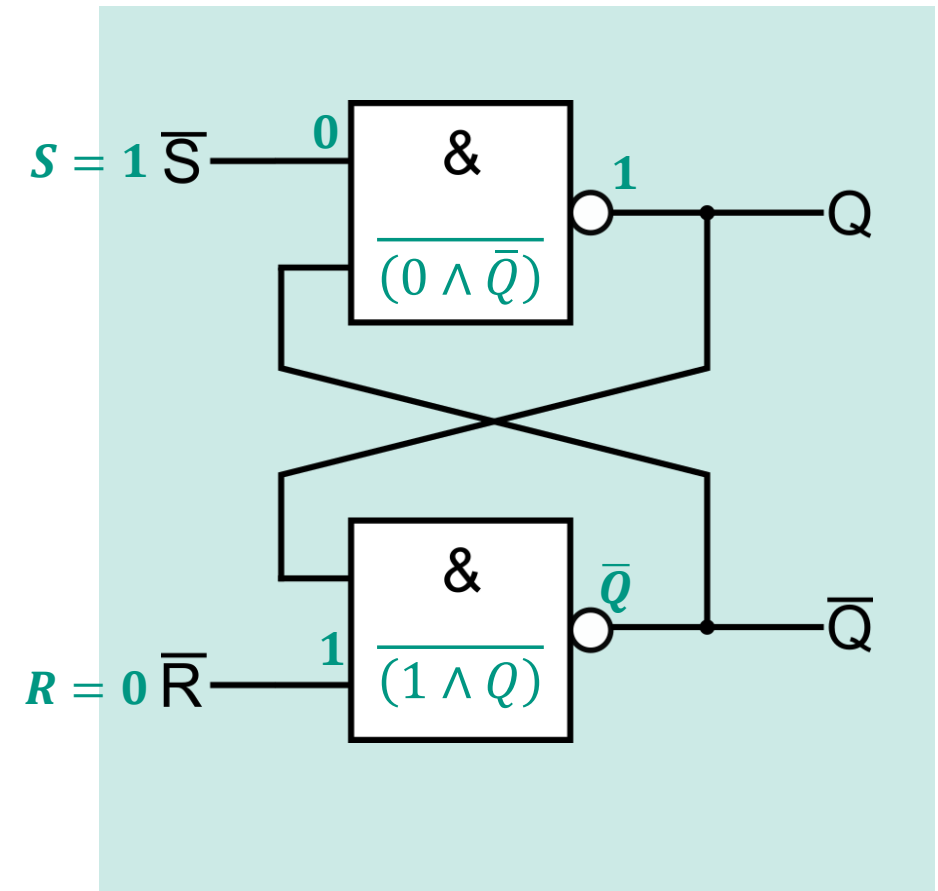
Ungetaktetes RS – FlipFlop

S	R	Q
0	0	Q
0	1	0
1	0	1
1	1	- (1)



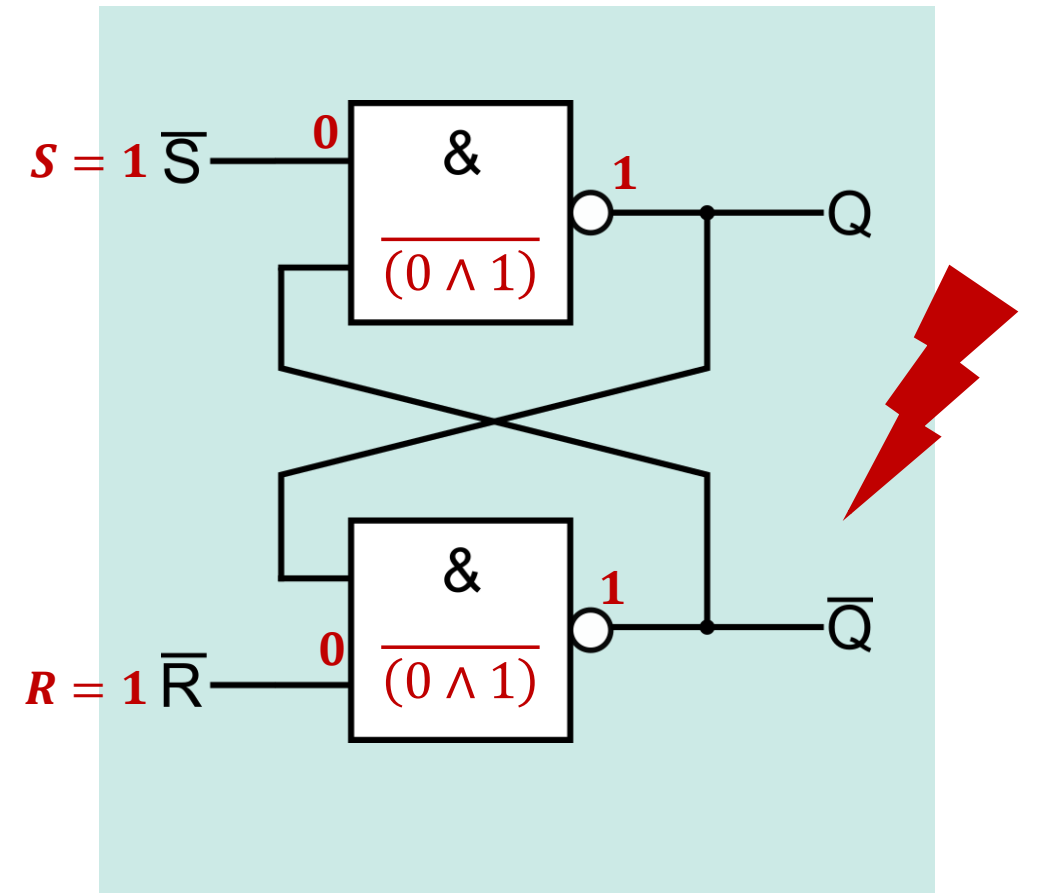
Ungetaktetes RS – FlipFlop

S	R	Q
0	0	Q
0	1	0
1	0	1
1	1	- (1)



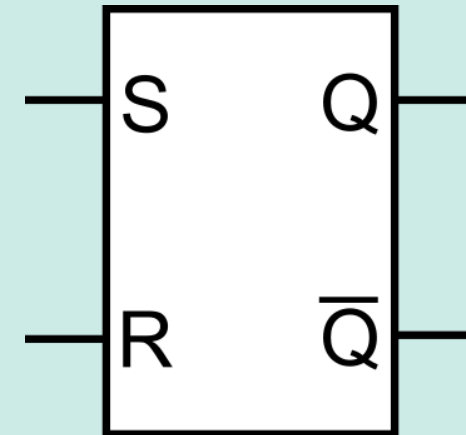
Ungetaktetes RS – FlipFlop

S	R	Q
0	0	Q
0	1	0
1	0	1
1	1	- (1)



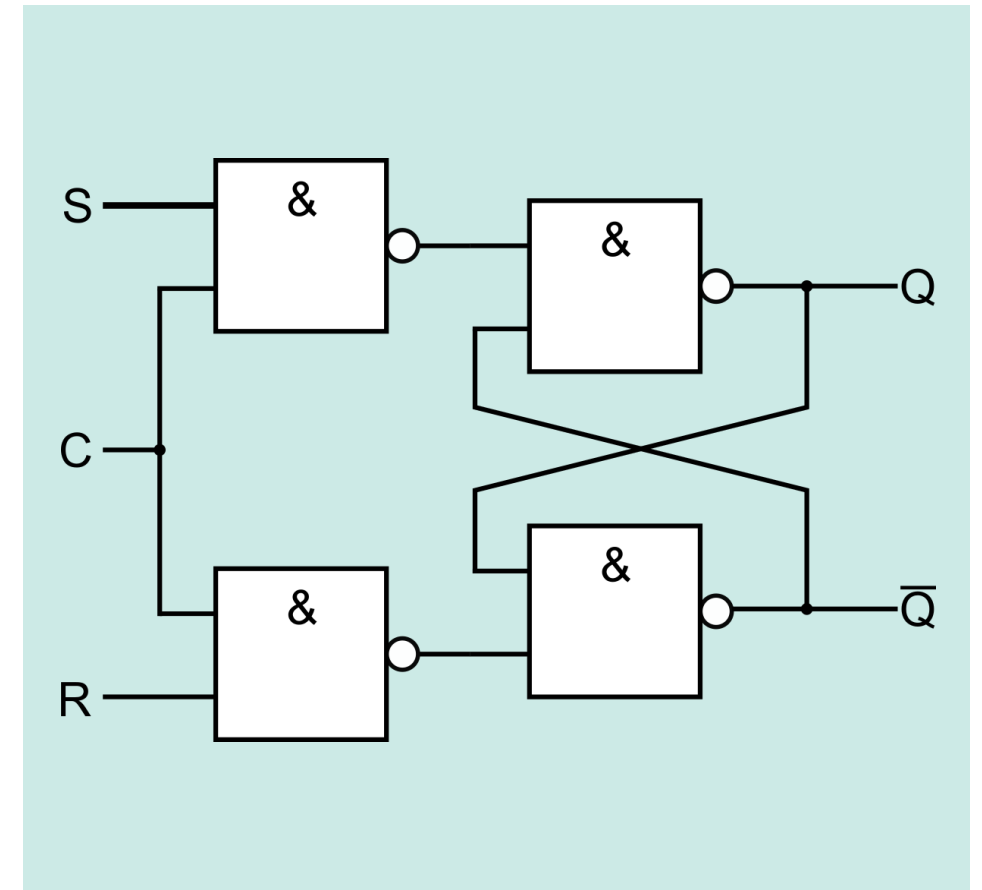
Ungetaktetes RS – FlipFlop

S	R	Q	Funktion
0	0	Q	speichern
0	1	0	rücksetzen
1	0	1	setzen
1	1	- (1)	verboten



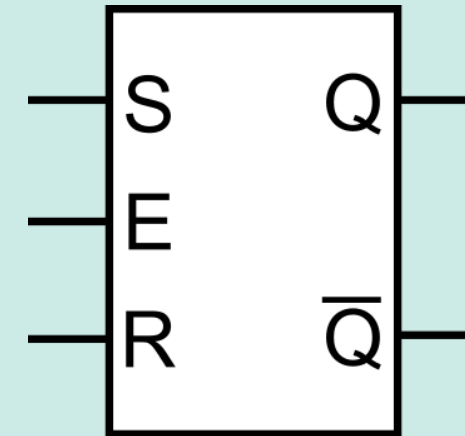
RS-Flipflop mit Taktpegelsteuerung

C	S	R	Q	Funktion
0	X	X	Q	unverändert
1	0	0	Q	unverändert
1	0	1	0	rücksetzen (0 schreiben)
1	1	0	1	setzen (1 schreiben)
1	1	1	- (1)	verboten



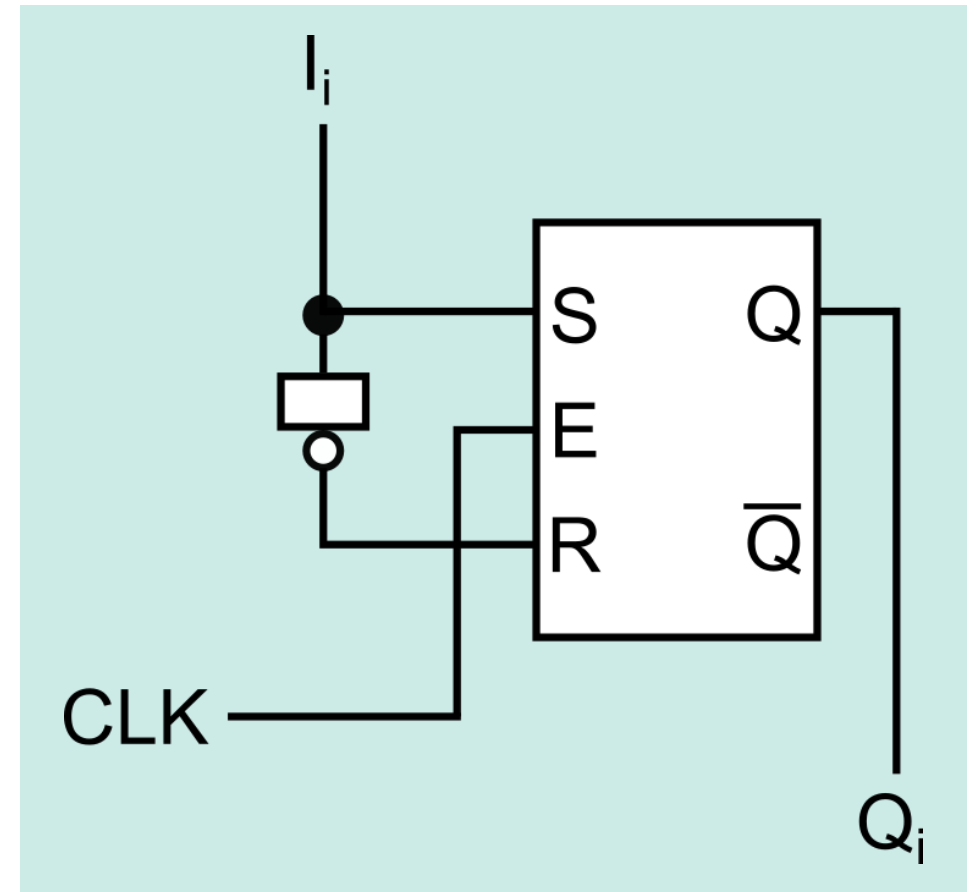
RS-Flipflop mit Taktpegelsteuerung

C	S	R	Q	Funktion
0	X	X	Q	unverändert
1	0	0	Q	unverändert
1	0	1	0	rücksetzen (0 schreiben)
1	1	0	1	setzen (1 schreiben)
1	1	1	- (1)	verboten

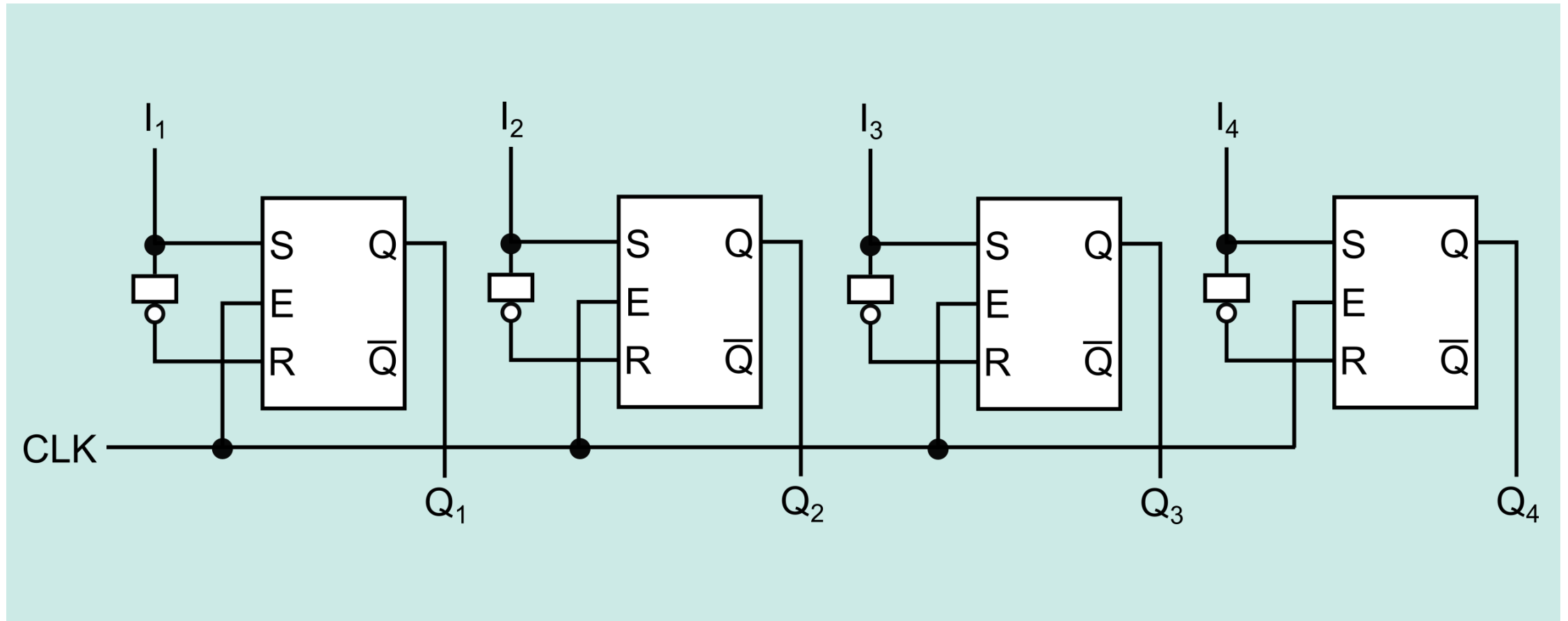


1-Bit Register

CLK	I_i	Q_i	Funktion
0	X	Q_i	unverändert
1	0	0	0 schreiben
1	1	1	1 schreiben

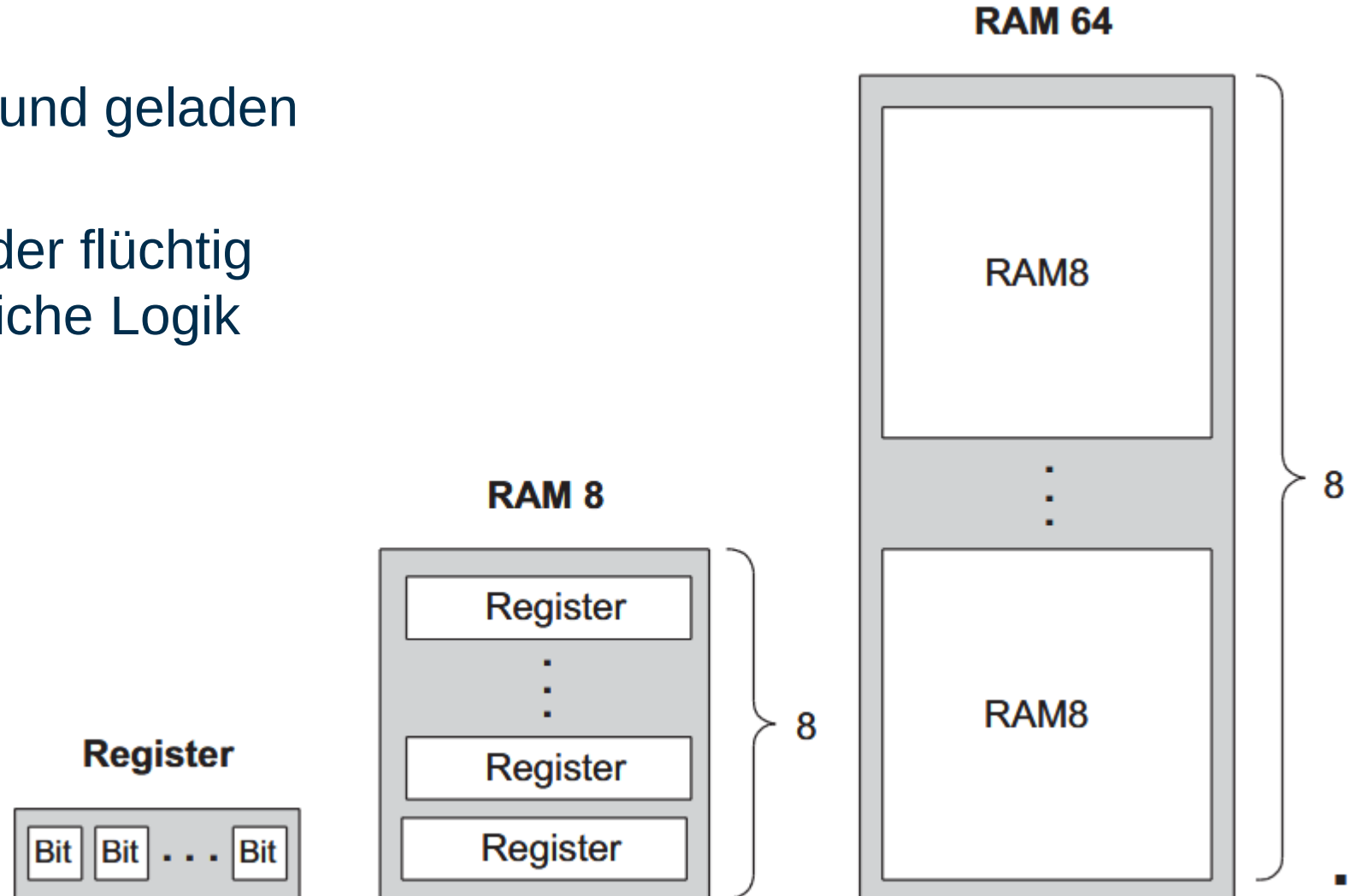


4-Bit Register



Speicher

- Daten können gespeichert und geladen werden
- Speicher kann dauerhaft oder flüchtig sein, aber hat stets einheitliche Logik



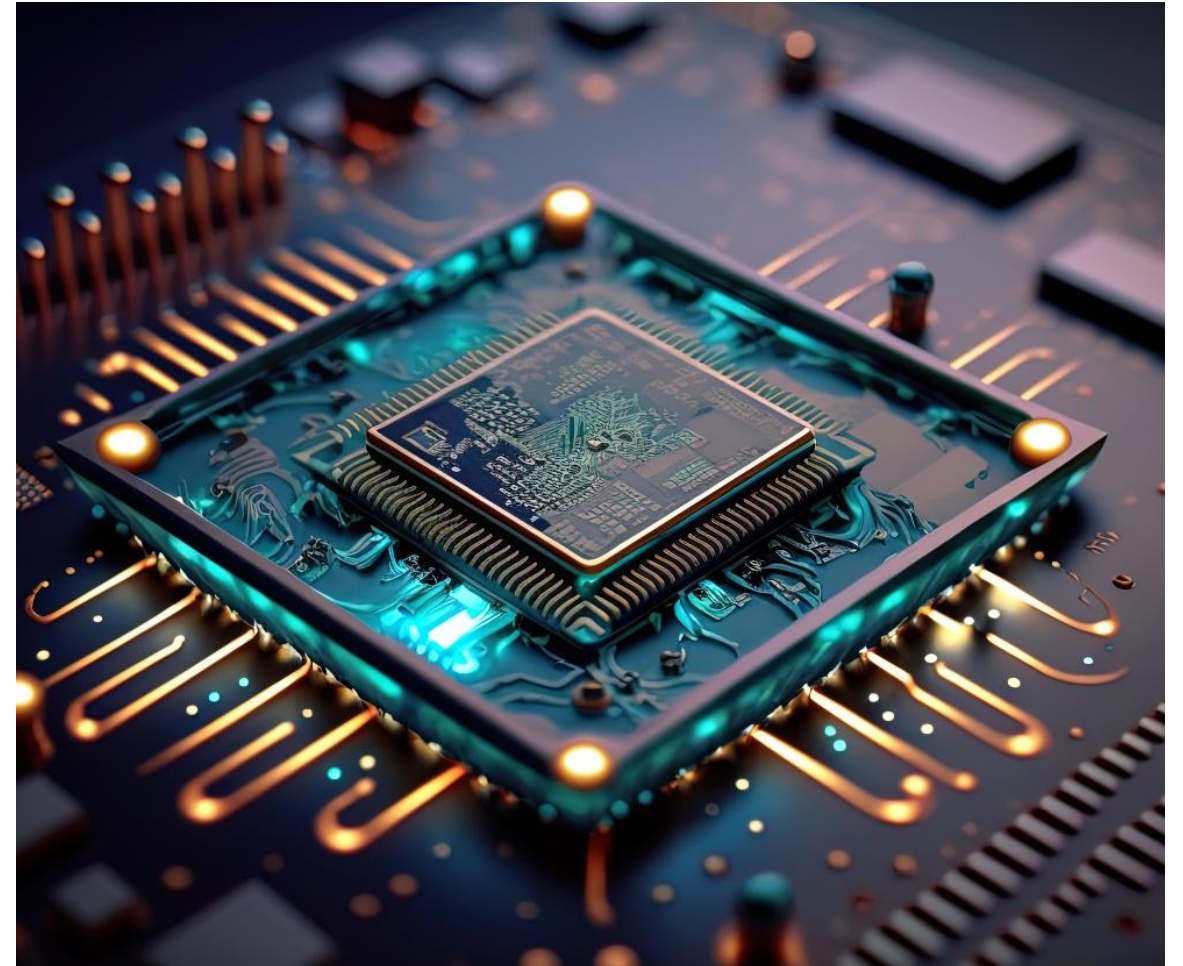
NAND Computer

- Architektur
 - Akkumulator
 - vollständige ALU
 - 8-Bit-Register
 - getrennte RAM/ROM-Bereiche (Harvard-Architektur)
 - Sprungvorhersage
 - Interrupts
 - 96 Taktzyklen für eine Instruktion
 - 100kIPS (thousands of instructions per second)
 - Taktfrequenz 10MHz

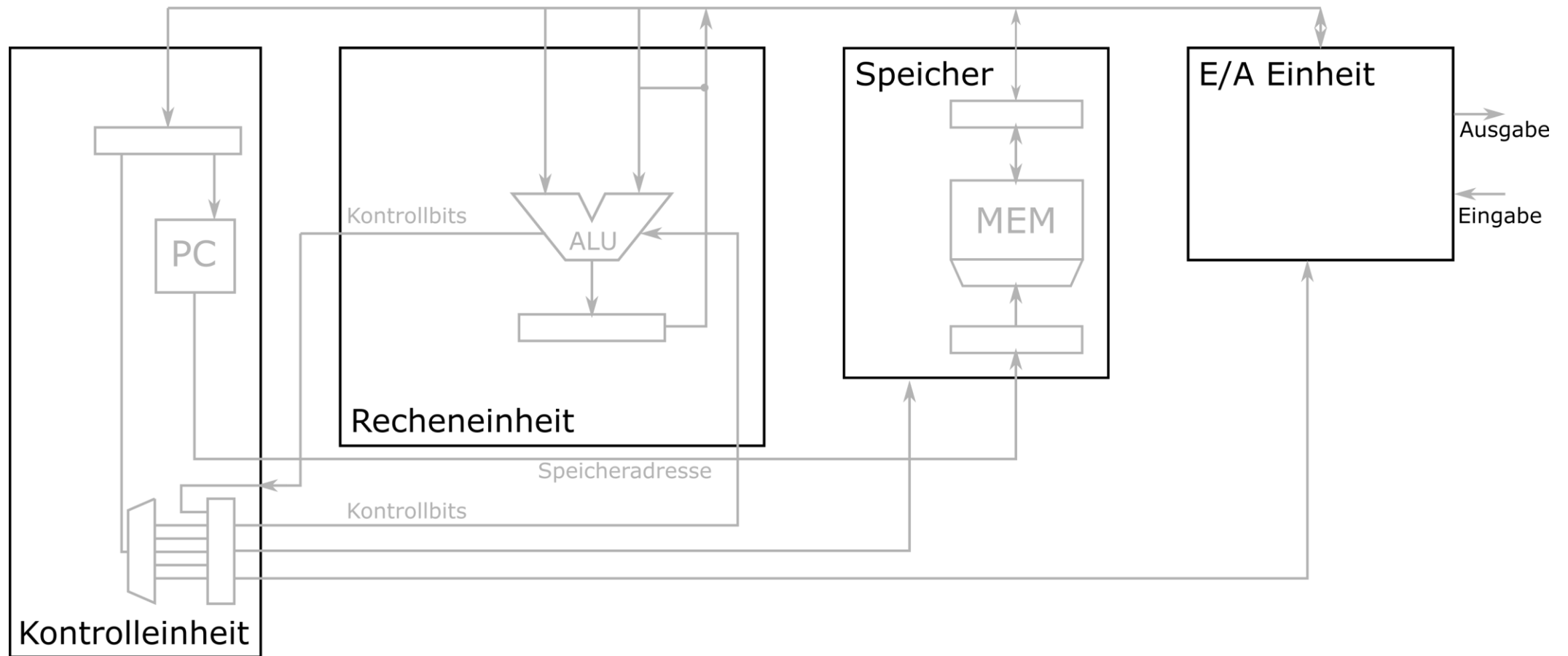


Von der ALU zum Prozessor

- Die ALU ist ein Rechenwerk
 - Befehle können nacheinander in einer festen Reihenfolge bearbeitet werden
 - Operanden müssen bei der Erstellung der Befehle bekannt sein
- Wie kann die ALU flexibler genutzt werden?
 - Werte aus Speicher lesen
 - Werte abspeichern
 - Bedingte Befehle, Abhängigkeiten

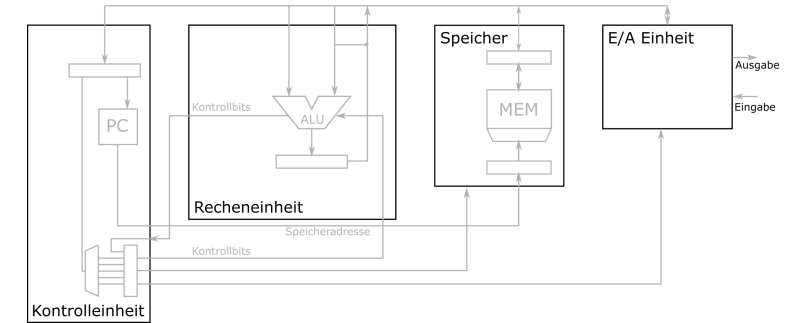


Aufbau eines einfachen Prozessors



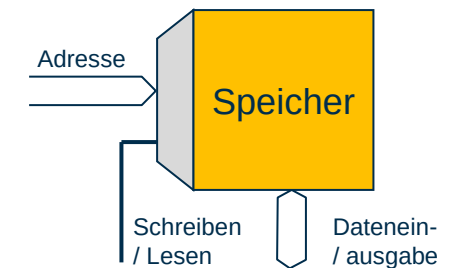
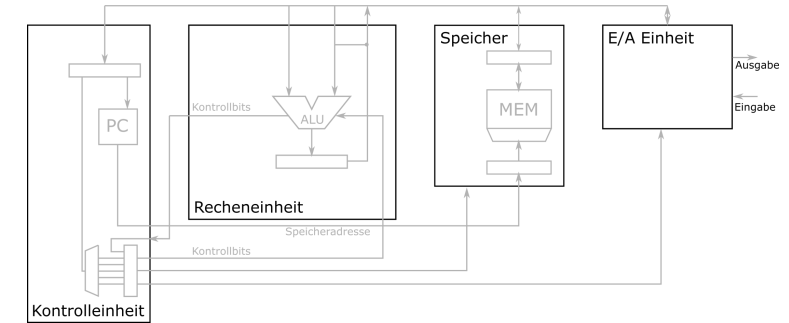
Kontrolleinheit

- Koordination der Komponenten des Prozessors
 - Versorgung mit Daten und Adressen
 - Selektion von Einheiten
 - Auswahl von Operanden und Operationen
- Ursprünglich Steuerung durch den Bediener, später Automatisierung durch zeitliche Abfolge von Binärinformationen
- Formalisierung der Anweisung nötig
 - Befehl: (Maschinenlesbare) Anweisung, aus der hervorgeht, welche Operation mit welchen Operanden durchgeführt werden soll
 - Befehlsformat: Festlegung, wie die für einen Befehl notwendigen Angaben dargestellt werden



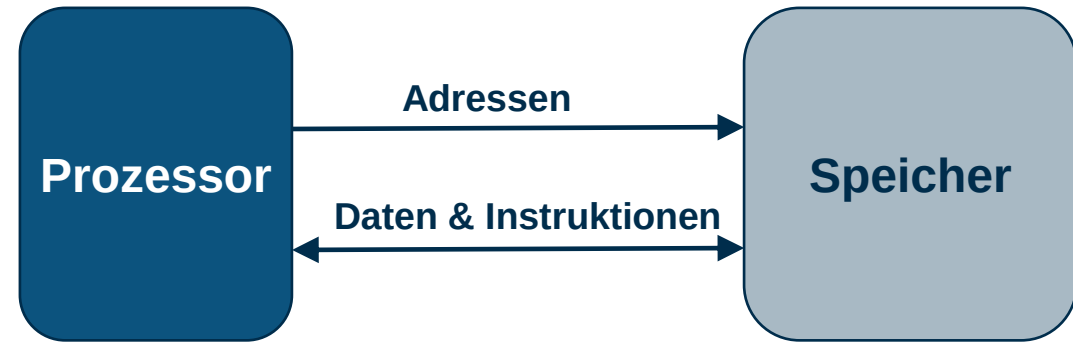
Speicherwerk

- Im einfachsten Fall nur eine Reihe von Registern zur Aufbewahrung von Operanden, Bedingungsvektor und Ergebnis.
- I.A. jedoch ein mehr oder weniger umfangreicher, adressierbarer Schreib/Lese-Speicher.
- Ermöglicht es, während der Verarbeitung eine große Zahl von Operanden und (Zwischen-) Ergebnissen bereitzuhalten, um so nicht für jede Operation das Ein-/Ausgabewerk benutzen zu müssen.
- Die Zahl der verfügbaren Speicherzellen richtet sich nach der Breite des Adressvektors.

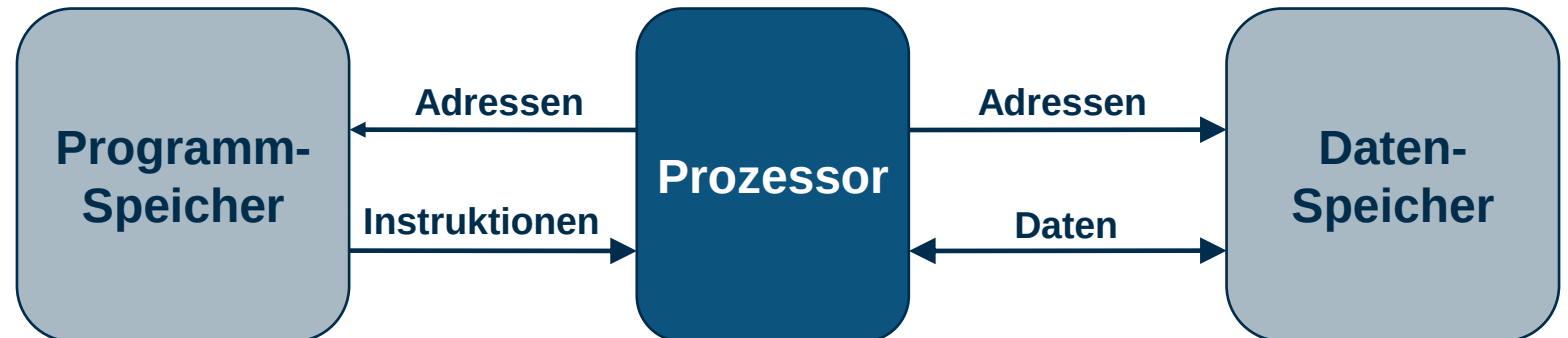


Speicheranbindung

- Von Neumann-Architektur:

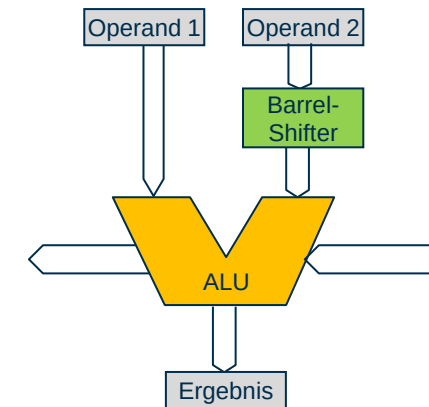
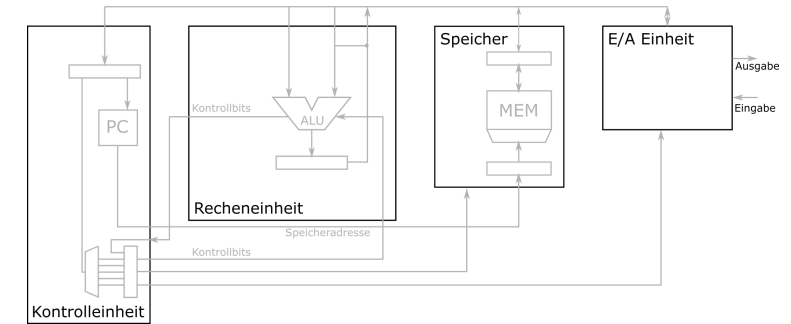


- Harvard-Architektur :



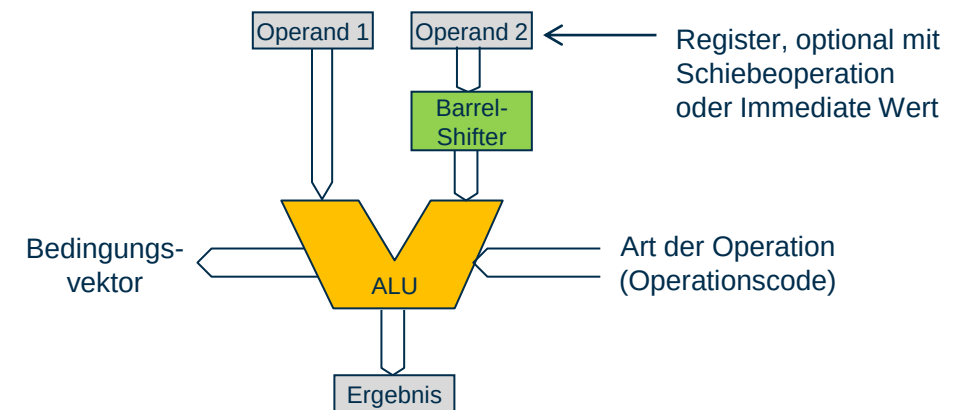
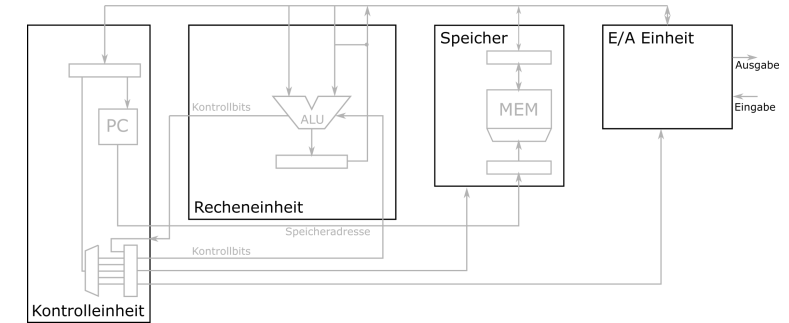
Recheneinheit

- Verarbeitung der Daten
- Stellt einen Satz von Basisfunktionen aus dem Bereich der numerischen und logischen Verknüpfungen zur Verfügung
- Beispiele für ein einfaches Rechenwerk:
 - Arithmetisch
 - Addition, Subtraktion
 - Logisch
 - Konjunktion, Disjunktion, Vergleich
 - Negation
 - Komplementbildung
 - Links-/Rechtsverschiebung
 - Links-/Rechtsrotation



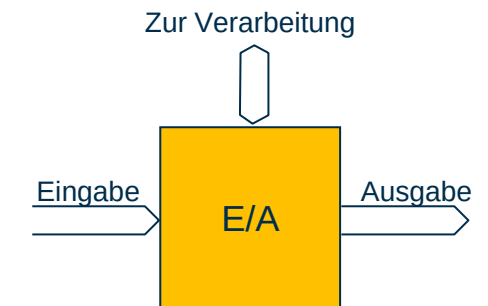
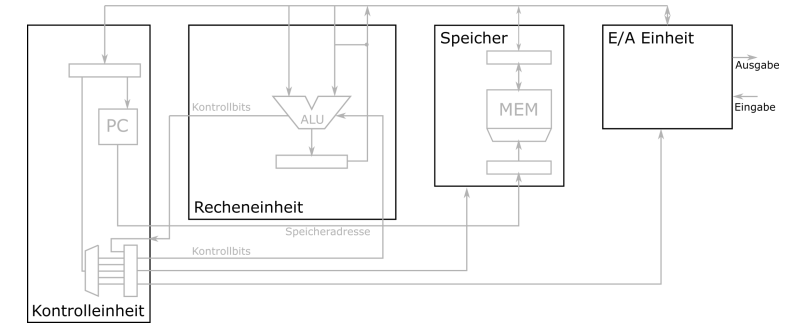
Arithmetisch-logische Einheit

- Üblicherweise Beschränkung auf Funktionen mit zwei Operanden
 - gegebene maximale Stellenzahl
 - Genauigkeit (Bitbreite der Architektur)
- Einzelne Merkmale eines Operanden oder Ergebnisses können als Binärwerte in einem sogenannten zur Verfügung gestellt werden
 - Bedingungsvektor („flags“)
 - Carry, Negative, Zero, Overflow



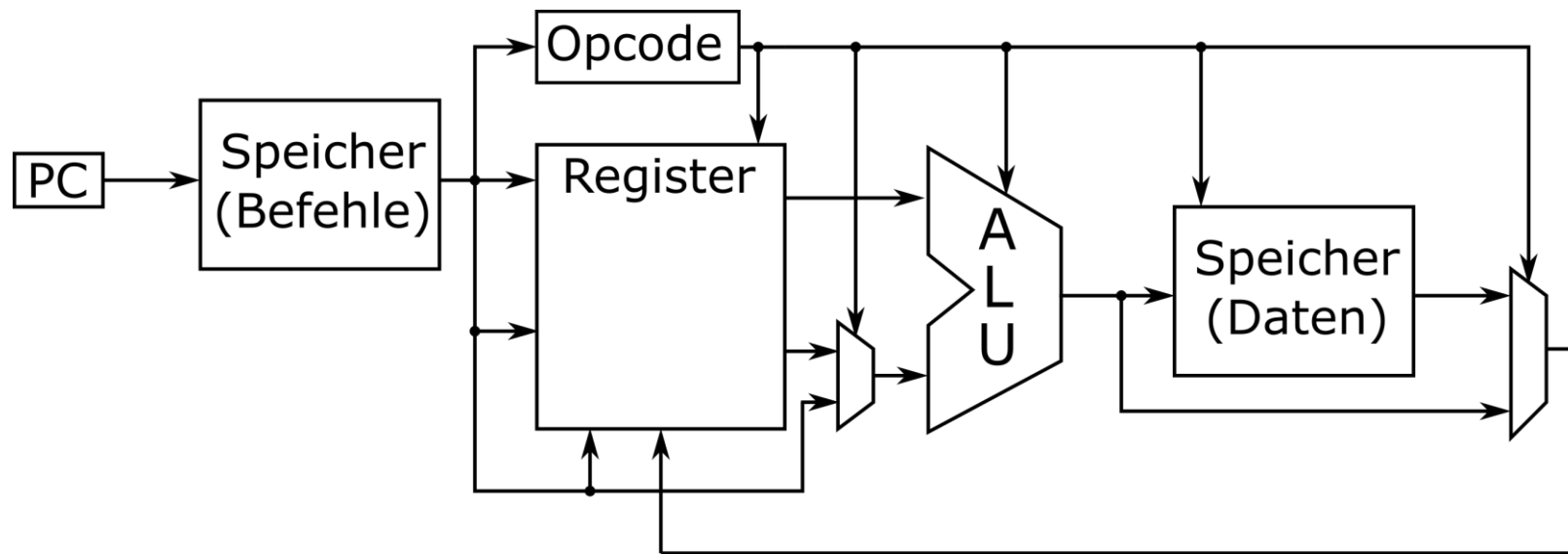
Eingabe-/Ausgabewerk

- Kommunikation mit dem Benutzer
- Eingabe
 - Datenübernahme von extern angeschlossenen Geräten
 - Umsetzung in eine geeignete, normierte Darstellung zur weiteren Verarbeitung
- Ausgabe
 - Aufbereitung des Datenformats
 - Weiterleitung an externe Ausgabegeräte
 - Eventuell Auswahl des Ausgabegerätes



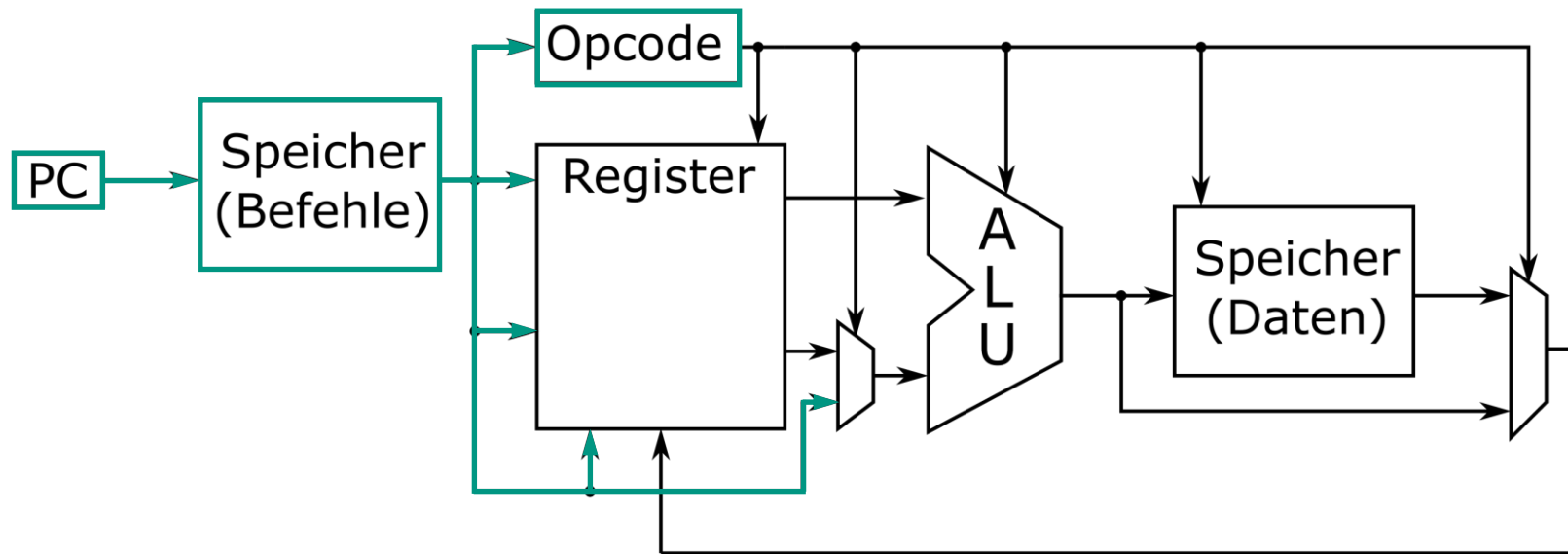
Ein einfacher Prozessor – Ablauf eines Befehls

- Ein einfacher Prozessor – Ablauf eines Befehls



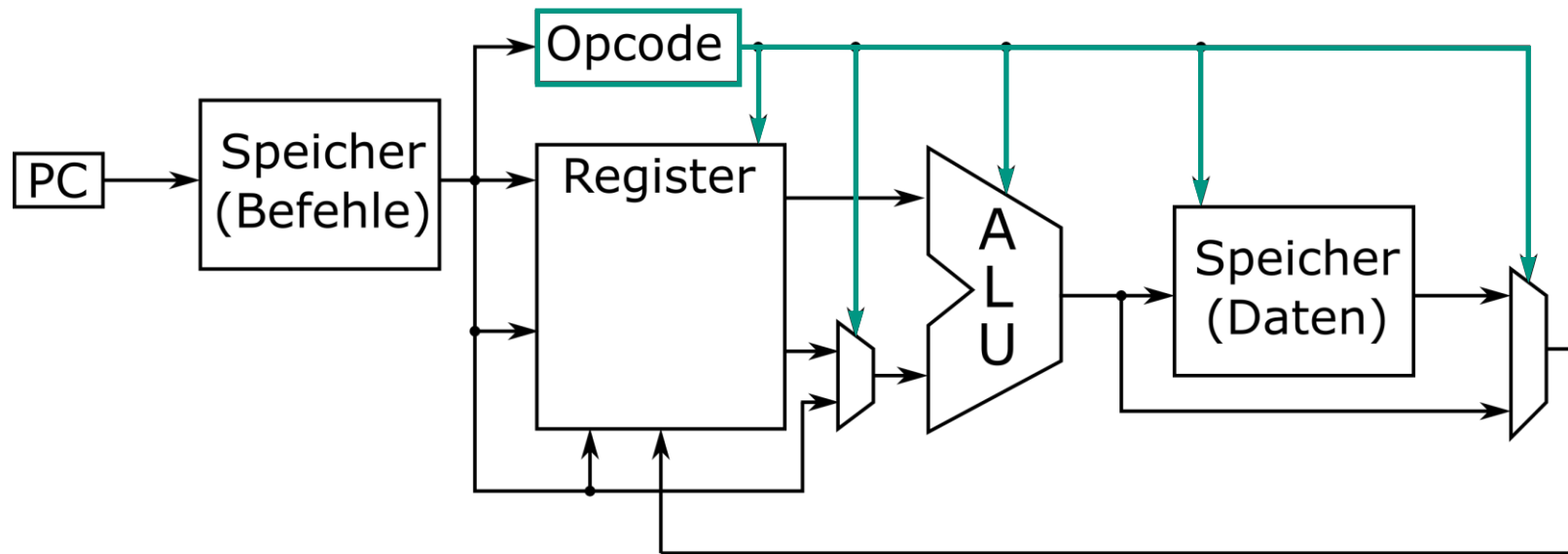
Ein einfacher Prozessor – Ablauf eines Befehls

- 1. Schritt: Laden des Befehls aus dem Speicher (FETCH)



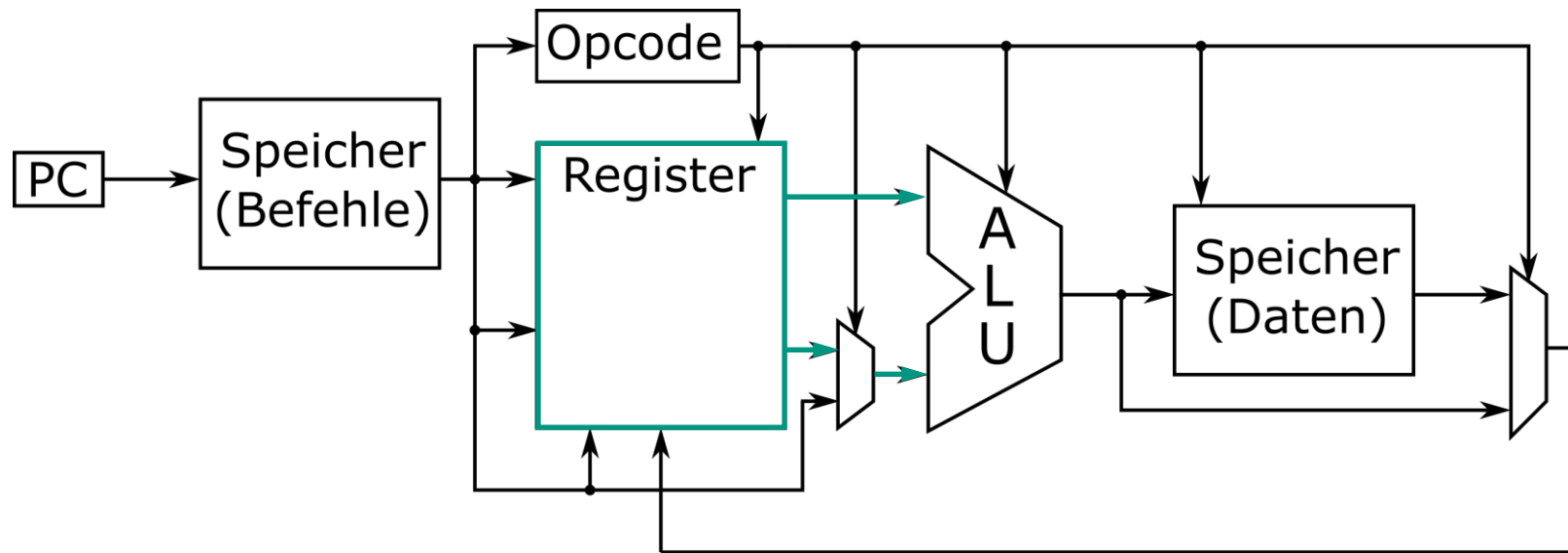
Ein einfacher Prozessor – Ablauf eines Befehls

- 2. Schritt: Dekodieren des Befehls (DECODE)
 - Welcher opcode, was muss getan werden?



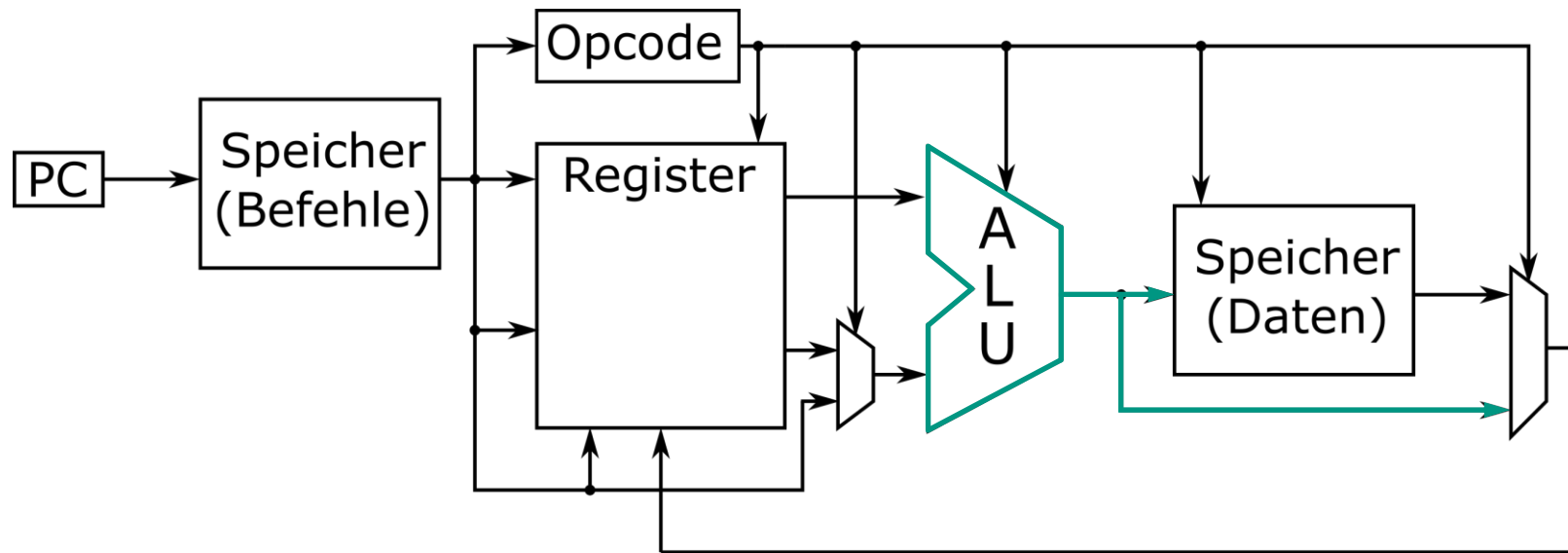
Ein einfacher Prozessor – Ablauf eines Befehls

- 3. Schritt: Operanden laden (FETCH OPERANDS)
 - Welche Operanden werden benötigt?



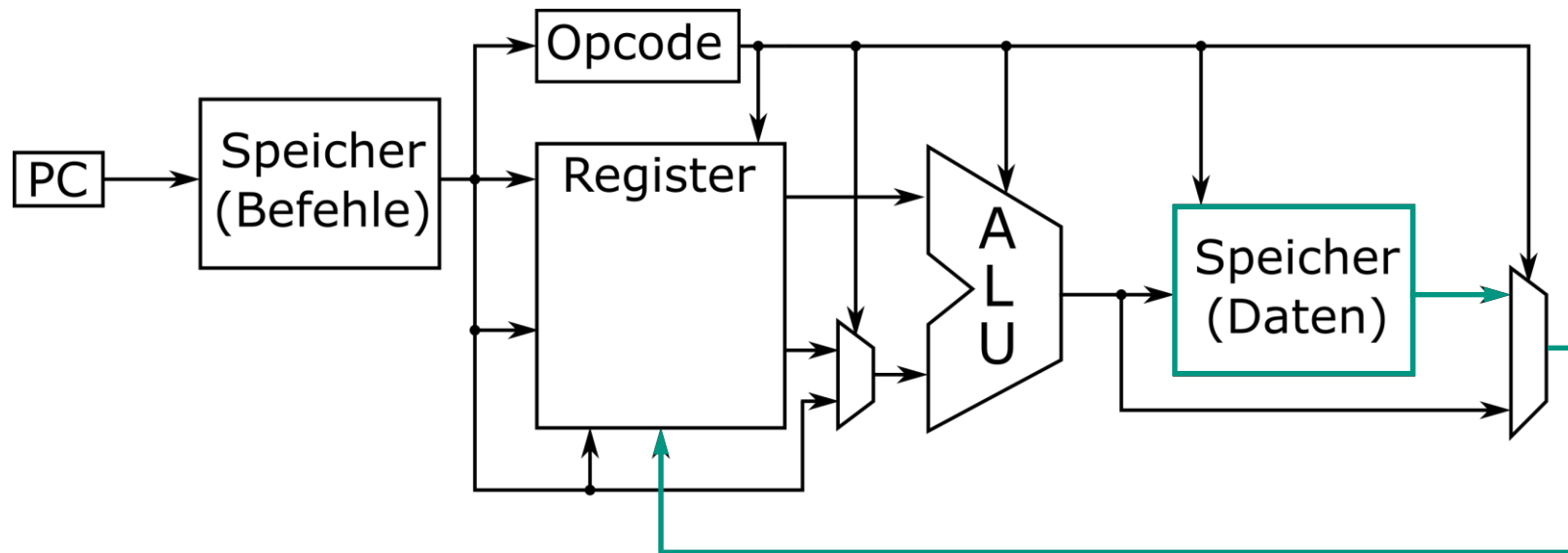
Ein einfacher Prozessor – Ablauf eines Befehls

- 4. Schritt: Eigentliche Berechnung ausführen (EXECUTE)
 - Was soll berechnet werden?



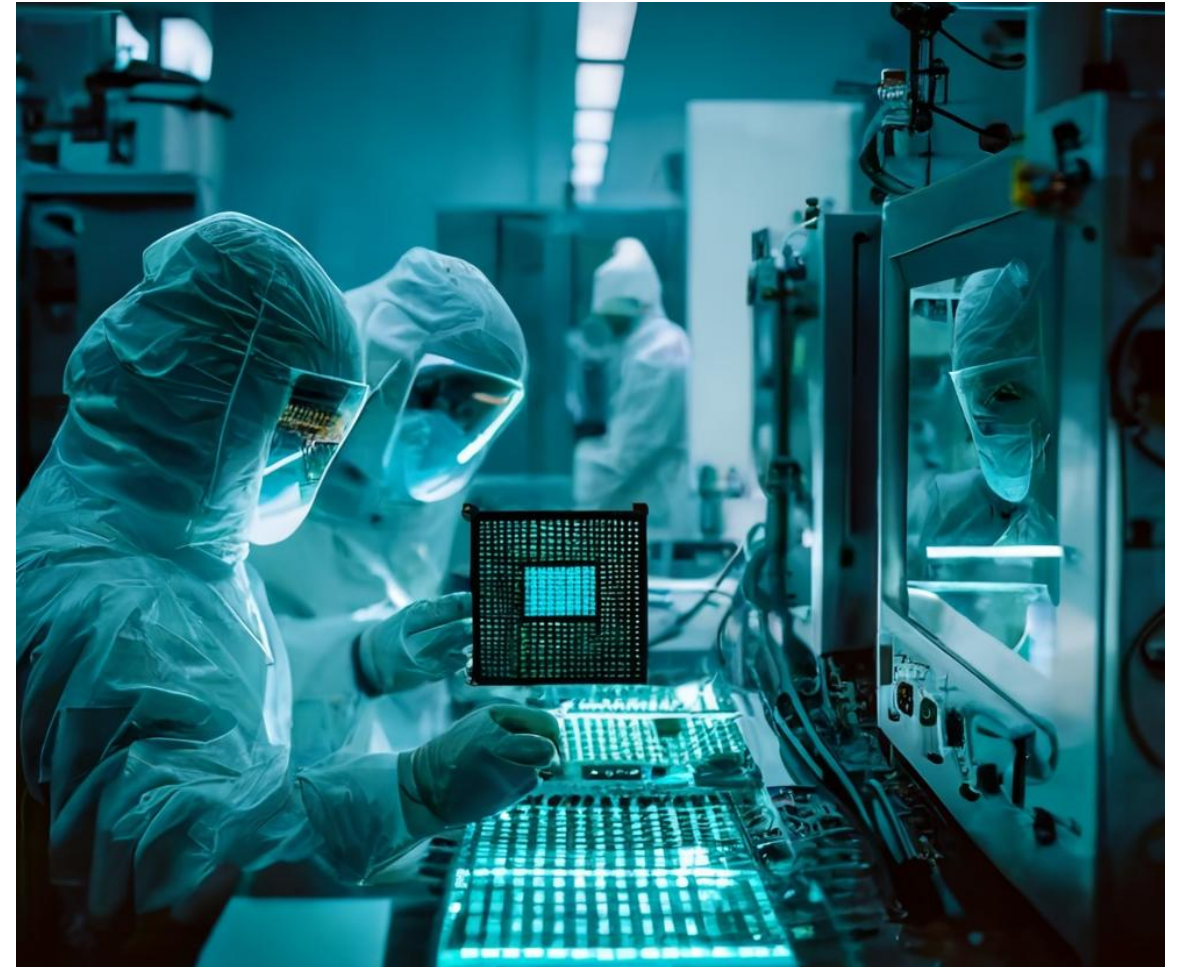
Ein einfacher Prozessor – Ablauf eines Befehls

- 5. Schritt: Ergebnis abspeichern (WRITE BACK)
 - Was soll mit dem Ergebnis passieren?



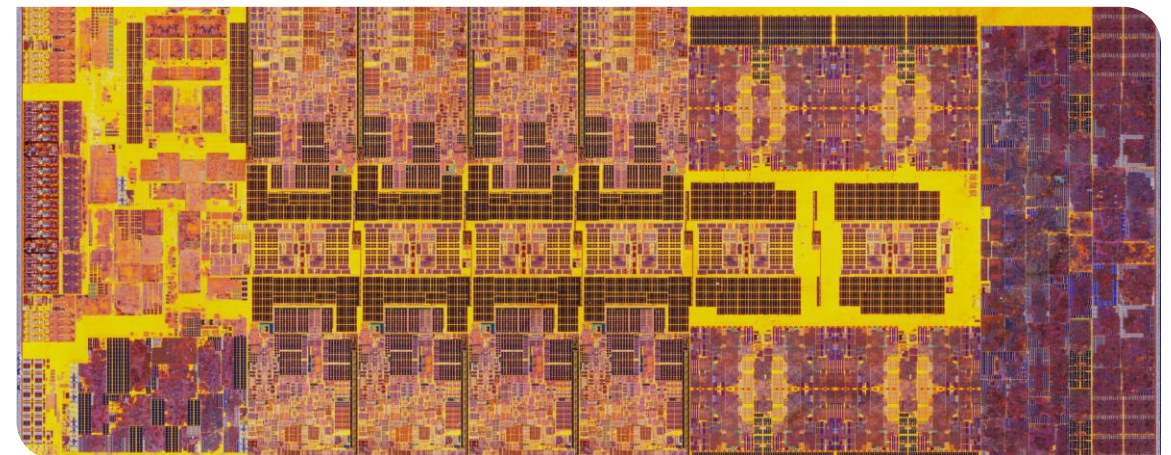
Aktuelle Prozessoren

- Die ALU ist auch heute immer noch einer der „Rechenkerne“ eines Prozessors
- Es gibt sehr viele ALUs und noch viele weitere „Spezial-Einheiten“
- Und noch viele andere Optimierungen!



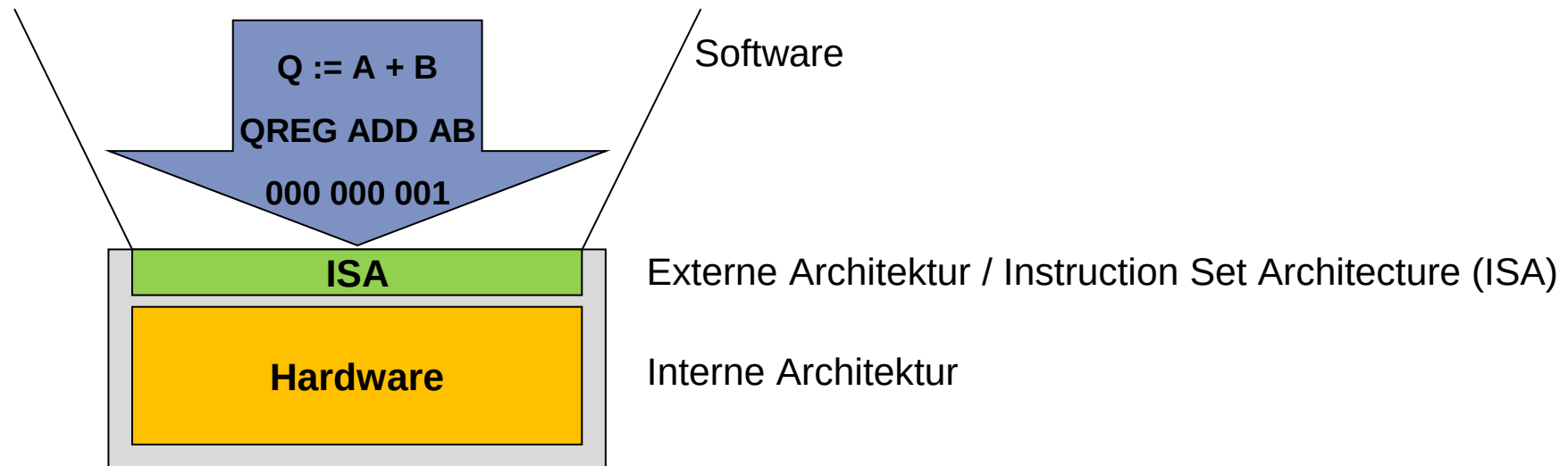
Beispiel Intel i9 13th Generation

- Viele verschiedene Kerne
 - 8 Leistungsstarke Performance-Cores(P-cores)
 - 16 Efficient-Cores (E-cores)
- Viele ALUs auch mit MUL/DIV
- Vector ALUs
- Anbindung an Speicher immer wichtiger



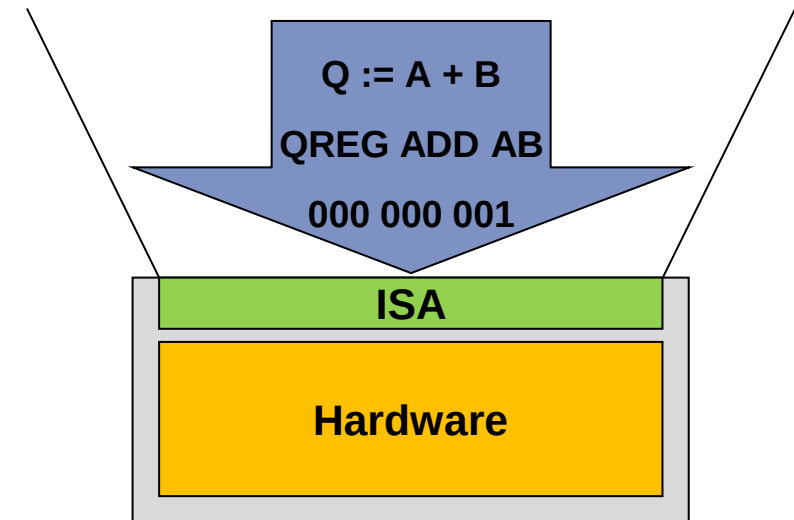
Befehlssatzarchitektur

- Instruction Set Architecture (ISA)
- Nach außen sichtbare Architektur eines Prozessors
- Schnittstelle zwischen Software und Hardware
 - vollständige Abstraktion der Hardware



Befehlssatzarchitekturen

- IA-32 (Intel Architecture 32)
 - Ursprünglichen x86-Architektur (16-Bit)
 - Erweiterung des Befehlssatzes auf 64-Bit x64
- IA-64 (Intel Architecture 64)
 - Itanium-Architektur
 - Mit x86 bzw. IA-32 inkompatibel
- ARM
 - Effizienter Befehlssatz, gut für Optimierungen im Bereich der Ausführungsgeschwindigkeit und der Stromaufnahme
- RISC-V
 - Offener Standard



Verschiedene Typen von Befehlssätzen

- Complex Instruction Set Computing (CISC)
- Reduced Instruction Set Computing (RISC)
- Very Long Instruction Word (VLIW)
- Explicitly Parallel Instruction Computing (EPIC)

Diskussion RISC versus CISC

- Unterscheidung RISC/CISC ist für moderne CPUs zu „einfach“
 - Früher: x86 versus ARM
- Prozessknoten, Mikroarchitektur und ISA haben je einen großen Einfluss auf die Leistung eines Chips
- CISC- und RISC-CPU's haben sich seit Jahrzehnten einander angenähert
 - Moderne x86 Architekturen verfügen über "RISC-ähnlicher" Dekodierungsmethoden
 - ARM: Multicycle instructions

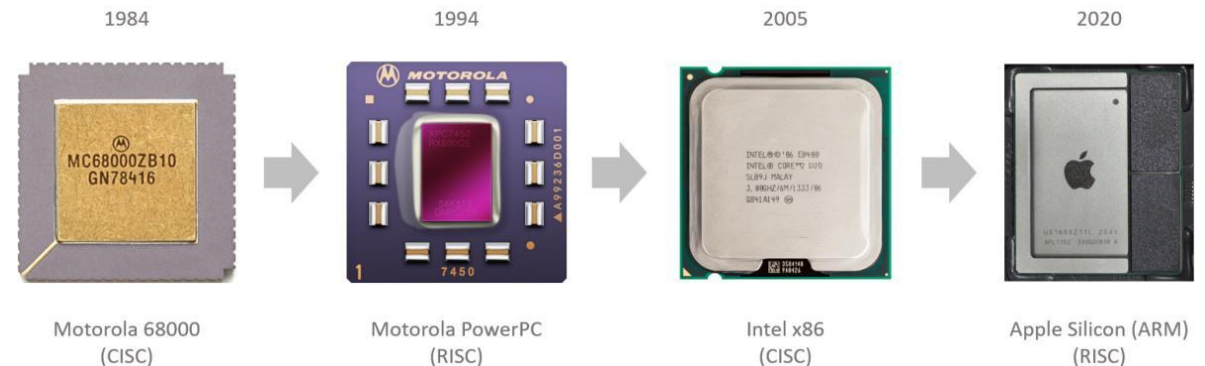
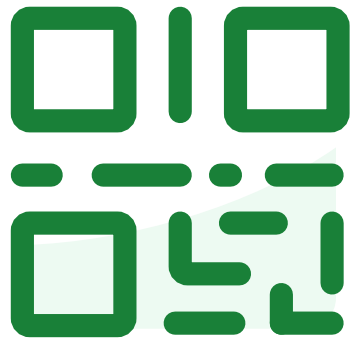


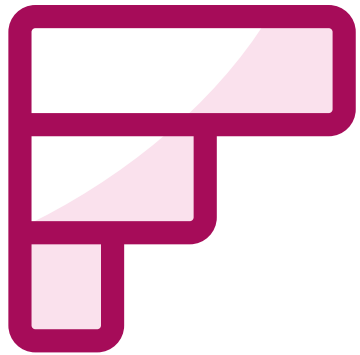
Image: <https://www.epcc.ed.ac.uk/whats-happening/articles/evolution-risc-architecture>



**Join at slido.com
#2719636**



Welche Aussagen treffen zu?



Wie wird ein Befehl in einem einfachen Prozessor schrittweise abgearbeitet?



**Was war heute neu für Sie, bzw.
hat sie überrascht?**

